

密级：_____

浙江大学

硕士学位论文



论文题目 动态体积云的建模、实时
渲染及移动端加速方法

作者姓名 兰未

指导教师 童若锋教授

学科(专业) 计算机技术

所在学院 计算机科学与技术

提交日期 2019-12-15

A Dissertation Submitted to Zhejiang
University for the Degree of
Master of Engineering



TITLE: Dynamic volumetric cloud
modeling, real-time rendering, and
acceleration method on mobile

Author: Wei Lan

Supervisor: Prof. Ruofeng Tong

Subject: Computer Technology

College: Computer Science and Technology

Submitted Date: 2019-12-15

摘要

在计算机图形学领域中，自然景物模拟是重要的研究课题之一，而云的模拟是自然景物模拟中十分重要的一部分。随着计算机性能的提高，人们对于模拟的要求也随之提高，云的模拟要具有高逼真、实时性、动态和可交互的特点。云的模拟分为建模和渲染两部分，其中建模部分构建描述云的特征的模型；渲染部分根据建模，计算光照和阴影，将云在屏幕上绘制出。云的模拟方法分为基于物理模型和基于经验模型两种，前者模拟云内部真实的物理活动，效果逼真但是计算量庞大，无法实时渲染；后者借助于数学模型，对云的特征进行归纳建模，可以实时渲染，效果的好坏取决于模型本身和后续渲染，一个好的建模和渲染方法对于云的实时模拟至关重要，因此本文提出了一种基于体绘制的云的建模和渲染方法。

本文的主要工作内容有三个方面：

一是提出一种基于噪声生成的体积云建模的方法。该方法基于 Perlin 噪声和 Worley 噪声生成的分型噪声，同时结合密度-高度函数和时间函数生成体积场，对云进行建模，具有动态、高逼真性，可以同时用于多种类型的低云族。

二是提出一种基于光线步进的体积云渲染的方法。该方法结合比尔-朗伯定律、相位函数、暗边效应等计算光照，使用光线步进的优化算法对云进行实时渲染，云的效果逼真，光照更加真实，具有实时性，可以模拟云的运动，可以穿过云层产生交互。

三是提出一种体积云在移动端加速的方法。该方法基于模型对云的体积场生成密度估计函数和基于 Value 噪声生成的湍流噪声对体积云的体积场进行建模，使用基于模型表面的光线步进的优化算法进行渲染，使用 Shadow Map 加速阴影渲染，在移动端实现较为逼真的动态体积云的实时渲染，可以模拟云的运动，可以穿过浅层层层产生交互。

关键词 体绘制；体积云；光线步进；噪声；实时渲染；

Abstract

In the field of computer graphics, natural scene simulation is one of the important research topics, and cloud simulation is an important part of natural scene simulation. With the improvement of computer performance, people have higher and higher requirements for simulation. Cloud simulations must be highly realistic, real-time, dynamic, and interactive. Cloud simulation is divided into two sections: modeling and rendering. The modeling section will build a model that describes the characteristics of the cloud. The rendering section calculates light and shadow based on modeling and draws clouds on the screen. There are two cloud simulation methods: based on physical models and based on empirical models. The former simulates actual physical activities in the cloud and has realistic effects, but the amount of calculation is huge and cannot be displayed in real time. The latter uses mathematical models to summarize and construct the characteristics of the cloud. It can be rendered in real time. The quality of the effect depends on the model itself and subsequent renderings. A good model and rendering method is important for real-time simulation of the cloud. Therefore, this paper proposes a cloud modeling and rendering method based on volume rendering.

The main content of this article includes three aspects:

The first is to propose a volumetric cloud modeling method based on noise generation. This method is based on type noise generated by Perlin noise and Worley noise, and combines a density height function and a time function to generate the volume field to model the cloud. The cloud is dynamic and high-fidelity, and can be used for multiple types of low cloud etage at the same time.

The second is to propose a volumetric cloud rendering method based on Ray Marching. This method combines Beer-Lambert's law, phase function, and dark edge effects to calculate lighting. An optimized Ray Marching algorithm is used to render the cloud in real time. The effect of clouds is realistic, while the lighting is more realistic. The cloud movements and interactions can be simulated in real-time.

The third is to propose a method for accelerating volumetric cloud on the mobile. This method models the volume field of a volumetric cloud based on a density estimation

function generated by the polygon model and a turbulence noise generated by the Value noise. It uses an optimized Ray Marching algorithm based on the polygon model, and uses shadow maps to accelerate shadow rendering. A realistic dynamic volumetric cloud can be rendered in real-time on the mobile, which can simulate the movements of the cloud and the interactions with the shallow cloud.

Keywords Volume Rendering, Volumetric Cloud, Noise, Real-time Rendering, Ray Marching

目录

摘要.....	i
Abstract	iii
第 1 章 绪论	1
1.1 课题背景	1
1.2 国内外研究现状.....	2
1.3 本文研究内容.....	4
1.4 本文组织结构.....	5
1.5 本章小结	5
第 2 章 云的模拟相关技术	6
2.1 云的模拟概述.....	6
2.2 基于物理的渲染技术概述.....	7
2.3 双向散射分布函数概述	9
2.4 云的光照概述.....	11
2.5 高阶着色器语言概述	15
2.6 虚幻 4 引擎概述.....	16
2.7 本章小结	17
第 3 章 体积云建模.....	18
3.1 基于体纹理的体积云生成.....	18
3.2 基于 3D 噪声的体积云生成.....	20
3.2.1 基于 Perlin 噪声的雏形构建	22
3.2.2 基于分形噪声的细节构建.....	23
3.2.3 基于 Worley 噪声的低频细节构建.....	27
3.2.4 密度-高度函数.....	30

3.2.5 时间函数.....	31
3.3 实验结果	31
3.4 本章小结	33
第 4 章 体积云渲染.....	34
4.1 体积云光照计算.....	35
4.2 光线步进	39
4.2.1 光线步进原理.....	39
4.2.2 光线步进优化.....	42
4.3 实验结果	44
4.4 本章小结	49
第 5 章 体积云在移动端加速方法.....	50
5.1 超纹理	51
5.2 基于模型的密度估计函数.....	53
5.3 基于 Value 噪声的体积场扰动构建	55
5.4 基于模型的光线步进算法.....	58
5.5 基于 Shadow Map 的阴影渲染.....	59
5.6 实验结果	62
5.7 本章小结	67
第 6 章 总结和展望.....	68
6.1 本文工作总结.....	68
6.2 未来研究工作.....	68
参考文献	70
攻读硕士学位期间主要的研究成果.....	74
致谢.....	75

图目录

图 2-1 BSDF 的构成.....	10
图 2-2 光通量的外散射、吸收和内散射	11
图 2-3 Rayleigh 相位函数.....	13
图 2-4 云的平均统计相位函数	14
图 2-5 HG 相位函数.....	15
图 3-1 伪体纹理的构建.....	19
图 3-2 白噪声	21
图 3-3 Perlin 噪声生成方法.....	22
图 3-4 放大 8 倍的 Perlin 噪声	23
图 3-5 5 个不同 Octave 的 Perlin 噪声叠加	24
图 3-6 5 个不同 Octave 的 Perlin 噪声的绝对值叠加.....	25
图 3-7 5 个不同 Octave 的 Perlin 噪声的绝对值变换叠加	26
图 3-8 现实中自然景观的云	27
图 3-9 Worley 噪声	28
图 3-10 Worley 噪声取反	29
图 3-11 5 个不同 Octave 的 Worley 噪声叠加并取反.....	29
图 3-12 Perlin 噪声在 Worley 噪声上重映射	30
图 3-13 本文噪声结果.....	32
图 3-14 陈小龙 计算机技术与发展 2015 噪声结果.....	32
图 3-15 Schneider SIGGRAPH 2015 噪声结果	33
图 4-1 云中光的吸收示意图	35
图 4-2 使光线方向逆转的散射	37
图 4-3 距离场示意图.....	40
图 4-4 Ray Marching 示意图	41

图 4-5 基于模型生成的云视觉效果.....	44
图 4-6 基于体纹理生成的体积云视觉效果	45
图 4-7 本文实现体积云的视觉效果 1.....	46
图 4-8 《地平线：零之黎明》中云的视觉效果 1.....	46
图 4-9 现实自然景观中云的视觉效果 1.....	47
图 4-10 本文实现体积云的视觉效果 2.....	47
图 4-11 《地平线：零之黎明》中云的视觉效果 2.....	48
图 4-12 现实自然景观中云的视觉效果 2.....	48
图 5-1 放大 4 倍后的 Value 噪声	56
图 5-2 基于 Value 噪声生成的湍流噪声	57
图 5-3 Shadow Acne.....	61
图 5-4 移动端加速的体积云云海的视觉效果	64
图 5-5 《Sky 光·遇》中云海的视觉效果.....	64
图 5-6 移动端加速的体积云自阴影的视觉效果.....	65
图 5-7 《Sky 光·遇》中云的自阴影视觉效果.....	65
图 5-8 移动端加速的体积云翻涌的视觉效果	66
图 5-9 《Sky 光·遇》中云翻涌的视觉效果.....	66

第1章 绪论

1.1 课题背景

计算机图形学（Computer Graphics，简称 CG）是一种使用数学算法将二维或三维图形转化为计算机显示器的栅格形式的科学。计算机图形学是一门重要的计算机子学科，其主要的研究内容为计算机中图形的表示、计算、处理和显示，以及图形相关的算法和原理，它是计算机科学中最重要的分支之一。近年来，随着科技的发展和计算机技术的进步，计算机图形学和我们的生活息息相关，它在各个领域和方面都有着重要的作用，包括图形图像以及视频的处理、工业界中模型的创建、各个平台中游戏的制作、生物信息的提取等等。

在计算机图形学领域中，自然景物的模拟是重要的研究课题之一，其在虚拟现实^[1]、视景仿真、游戏、影视、广告等领域有着广泛的应用。云是自然景观中必不可少的一部分，它在天空中以各种各样的形状存在着，给世人带来无可或缺的视觉体验与美感和无尽的遐想。因此在自然景物的模拟里，云的模拟和渲染是十分重要的一部分。如果缺少云的模拟，将会显著的影响自然场景的真实感和协调性。但是因为云无论在空间上还是时间上都是不规则且不重复的，且涉及到内部光线散射和吸收，所以云的模拟一直是一个具有挑战性的研究课题。

云的模拟在技术实现上通常分为建模和渲染两部分，其中建模部分需要根据云的特征对云进行建模；渲染部分需要根据建模，计算光照和阴影，并最终将云在屏幕上绘制出来。而云的模拟方法可以分为基于物理模型方法和基于经验模型方法两大类。基于物理模型的方法是对云内部真实的物理活动进行分析，模拟云的运动的过程。这种方法生成的云具有高逼真性但是计算量十分庞大，渲染速度无法达到实时的要求。基于经验模型的方法主要借助于数学模型，通过对云的特征进行归纳从而对云进行建模，相较于物理方法有较好的实时性，通过经验模型所得到的构建结果是可控的，其效果的好坏取决于模型自身以及后续渲染方法，

而一个好的建模和渲染方法对于云的实时模拟至关重要。

随着科技的发展,在虚拟现实、三维游戏等领域中,越来越多地应用了实时渲染技术。实时渲染指的是根据图形学算法将三维数据绘制到二维位图之中,并将这些位图实时显示。它的本质就是对图像数据的实时计算和输出,要求在短时间内渲染出一张图片,并显示出来,同时渲染并显示下一张图片。这种技术仅仅依靠 CPU 是没办法完成的,还需要依靠 GPU 实现。相较之下云的实时渲染^{[2][3][4]}更加困难,因为云是由大量的粒子(小水滴、小冰晶等)随机组成的复杂系统,有着不规则的边缘和外表,而粒子的运动导致云的形状处于时刻变化中。因此云是动态的,时刻都在运动,并且运动轨迹比较复杂;云由大量粒子构成的,需要考虑云的各种物理光学属性;在大规模云海的模拟过程中,需要进行大量的计算,系统资源占用大。

近年来,随着计算机技术的发展和计算机性能的提高,人们对于虚拟环境的逼真性也随之提高。在例如大气海洋环境的模拟、飞行器模拟、计算机游戏以及影视科技等的应用中,云的模拟必须保证高逼真、实时性、动态和可交互的特点^[5]。随着移动端的普及和性能的快速发展,人们对于移动端的虚拟环境模拟的要求也随之提高。如何在移动端不具备 PC 端的高性能的情况下,同样能够在适当降低精度的情况下提升速度,使其和 PC 端一样能够实现大规模动态云的实时模拟,也是一个重要的研究课题。

基于上述背景,本文要完成的工作是对云进行建模和渲染,使其具有高逼真、实时性、动态和可交互的特点,并对其在移动端进行加速,在适当降低精度的情况下提升速度,使其和 PC 端一样能够实现动态云的实时模拟。

1.2 国内外研究现状

云的模拟技术是一个被广泛研究的领域,不断有许多新的想法和突破,来实现如何模拟出更逼真的云。

国外方面, Blinn^[6]在图像分析中使用了密度模型,并针对同一介质提出了低

反射率,单散射光照模型。随后 Blinn^[7]使用高度场和射线追踪技术对云进行建模。Perlin^[8]提出使用 Perlin 噪声对的云进行建模。Gardner^[9]使用空心椭圆体结合纹理函数技术对云进行建模。Dobashi^{[10][11][12]}等人使用粒子系统的方法对云进行建模,并将云空间划分为许多微小粒子,每个粒子都用一个立方体表示,并具有湿度、运动和粒子的三个属性。Bouthors^[13]等人提出了一个对层云的照明和着色技术,用来实现实时的逼真的渲染。作者通过使用最新的光照模型,他们计算了所有的光路并且使用基于物理相位函数保留了各向异性行为,最终实现以 18-40 帧每秒的效率渲染出逼真的云,这个技术只限于特定少数类型的云,其中最适合于层云的渲染。随后 Bouthors^[14]等人提出了一个多重各向异性散射技术,用于在云中进行交互,这个技术使用和前面类似的方法,最终实现以 2-10 帧每秒的效率渲染出具有非常真实的光照和细节的积云。Egor Ysov^[15]等人开发了一个新的渲染方法用于云的渲染。该技术使用预先计算好的光照和粒子来渲染逼真的积云,但是这个技术依赖于一个叫做像素同步的功能来实现体积感知混合,这个功能仅在 Intel HD 图形卡上可以使用。Schneider^{[16][17]}等人开发了一个新的云系统的,并在游戏《地平线:零之黎明》中使用,作者通过使用 Worley 噪声和 Perlin 噪声的组合以及光线步进技术设法在实时和动态光照的条件下渲染逼真的积云。Elhaddad^[18]等人提出了一种新的云的模拟方法,提出使用近似 Lennard-Jones 势能对云的运动进行简化和模拟。Nowak^[19]等人通过 UE4 引擎实现了体积云的建模和实时渲染,效果仍有改进空间。Hufnagel^[20]等人对云的渲染和照明的研究与开发做了全面的综述,在综述中,作者比较了不同的技术,并将他们互相对比和分析,其中涉及的一些技术包括: Billboards, Splatting, 体切片渲染和光线步进,探讨了不同技术所适合的云的场景和类型。Dobashi^[21]等人对云的建模,渲染和动画化方法进行了总结,并介绍了云的视觉模拟的相关研究。

国内方面,任威^[22]等人提出了一种实时模拟的方法,用于大规模三维云的模拟,论文中用 Navier-Stokes 流体力学公式和基于八叉树的模型化简策略对云进行建模,基于 Cell 的绘制策略,结合 Impostor 技术自动混合绘制对云进行渲染,该方法是基于物理的云的模拟方法,实现实时的逼真的渲染。陈小龙^[23]等人提出了

一种基于体绘制的模拟算法，用于动态三维云的模拟，论文中用 Perlin 噪声干扰椭球模型对云进行建模，用光线投射对体数据进行采样渲染，该方法可以模拟出的云的形状颜色位置的动态改变，也可以穿过云层，在云中进行交互。刘芳^[24]等人对体积云进行了研究和实现，论文中研究了基于粒子系统、气泡建模、体素建模、纹理映射、噪音函数的云的建模和基于 Perlin 噪音的体积云、基于细胞自动机理论的云的渲染，并对其进行比较和分析。柏靖云^[25]对动态体积云绘制进行了研究，实现球形粒子建模结合与计算光照渲染方法和噪声纹理采样结合比尔-朗伯定律光照模型方法，并对其进行比较和分析。

1.3 本文研究内容

本文研究体积云的实时渲染的实现和优化，具体工作主要有以下几个方面：

1、提出一种基于噪声生成的体积云建模的方法，本文在实现体积云建模的基础上，对体积云建模进行优化。该方法基于 Perlin 噪声和 Worley 噪声生成的分型噪声，同时结合密度-高度函数和时间函数生成体积场，对云进行建模。该方法建模出的云具有更多细节，更接近于现实中自然的云，同时可以对多种不同的云进行建模并且可以实现动态变化。

2、提出一种基于光线步进的体积云渲染的方法，在实现体积云渲染的基础上，对体积云渲染进行优化。该方法结合比尔-朗伯定律、HG 相位函数和云的暗边效应对体积云光照进行近似计算，使用改进的光线步进技术对云进行实时渲染。该方法渲染出的云效果逼真，光照更加真实，可以模拟云的运动，可以穿过云层产生交互。

3、提出一种体积云在移动端加速的方法，实现体积云在移动端的实时渲染。该方法基于模型对云的体积场生成密度估计函数和基于 Value 噪声生成的湍流噪声对体积云的体积场进行建模，使用改进的基于模型表面的光线步进算法进行渲染，使用 Shadow Map 加速阴影渲染。该方法可以在移动端实现较为逼真的动态体积云的实时渲染，可以模拟云的运动，可以穿过浅层云层产生交互。

1.4 本文组织结构

本文组织结构共分为六章，具体内容如下：

第一章绪论。这一章介绍了本文的课题背景、国内外研究现状、研究目标 and 内容。

第二章云渲染相关技术。这一章介绍了云渲染的相关技术与理论，包括云的模拟概述、基于物理的渲染概述、双向散射分布函数概述、云的光照概述、高阶着色器语言概述、虚幻 4 引擎概述。

第三章体积云建模。这一章首先介绍了根据体纹理生成体积云的方法和过程，然后介绍了根据 3D 噪声生成体积云的方法和过程，其中详细阐述了如何使用 Perlin 噪声和 Worley 噪声，以及结合密度-高度函数和时间函数生成体积云体积场的方法。

第四章体积云渲染。这一章通过比尔-朗伯定律、相位函数、暗边效应这些云渲染用到的光照函数和模型对光辐射度进行了近似的推导和计算，并详细阐述了结合上述光照模型使用光线步进算法生成体积云的方法，并分析了光线步进的优化方法，最后对结果进行了展示。

第五章体积云在移动端加速。这一章介绍了超纹理，然后详细了探讨基于超纹理概念使用模型对体积云体积场生成密度估计函数的方法，并与基于 Value 噪声生成的湍流噪声相结合完成对体积云体积场的建模，接下来在此基础上介绍了基于模型表面的光线步进加速算法，介绍了使用 Shadow Map 对阴影渲染的加速方法，最后对结果进行了展示。

第六章总结和展望。这一章对本文做了一个小结，对本文可改进的地方进行分析，并对未来的工作做展望。

1.5 本章小结

本章阐述了动态体积云实时渲染的背景与意义，并介绍了国内外云的渲染的研究现状。然后介绍了本文的研究内容，并介绍了本文的组织结构。

第2章 云的模拟相关技术

2.1 云的模拟概述

云的模拟实现大致可以分为建模和渲染两个部分，其中建模部分需要对云进行建模，渲染部分需要根据建模，计算光照和阴影，并最终将云在屏幕上显示出来。

云的模拟方法可以分为基于物理模型方法和基于经验模型方法两大类。

基于物理模型的方法是通过分析云内部真实的物理活动，从而模拟云的运动过程。这种方法生成的云非常逼真但是计算量十分庞大，对系统资源消耗巨大，渲染速度较慢，无法达到实时渲染的要求。基于物理模型的云建模方法主要有两种，分别是粒子系统模型和气泡模型。

粒子系统模型是用来模拟不规则物体的常用方法，该方法将云团看成是由一个个粒子所构成的实体，每一个粒子相当于云团中的一个小水滴，对粒子属性的描述就是对云层中水滴特征的描述，水滴的运动特征则通过粒子的运动模型来构建。但是因为云的细节描述比较复杂，因此粒子属性集的描述复杂，粒子系统的优越性随着属性集的复杂度增加而降低。在需要描述大量细节的时候，需要增加非常多的微小粒子，导致速度降低。

气泡模型是用一个气泡来模拟一个云团，气泡由层层重叠的曲面所构成，每一个气泡则由中心、半径、权重来描述。气泡建模方法的等效曲面可以被任意拉伸，可以利用多种渲染方式来绘制图像。用这种方法建模的好处是这些等效曲面被拉伸后，可以在建立云的大致模型后采用分形法给曲面添加细节描述。

基于经验模型的方法主要借助于数学模型，通过归纳云的特征来对云进行建模^{[26][27]}，通过经验模型构建所得到的结果是可控的，相较于物理方法有较好的实时性^[28]。基于经验模型的云建模方法主要分为三种，分别是平面云方法、模型云方法和体积云方法。

平面云方法不是用体而是用面来表示云，通过纹理映射的方法将纹理贴图在天空盒等面上显示图像。该方法相对简单，所得到的图像也比较简陋，无法体现云与光线的相互作用，也无法穿过云层产生交互。

模型云方法是用贴图加模型对云进行建模，通过读取预先制作好的模型和贴图来显示云。这种方法制作云的好处是方便艺术家控制，可以通过 PBR 得到较好的图像效果。但是这种方法不够灵活，而且做出来的效果固定，光影变化难以控制，对模型面数要求极高，模拟大规模云海时十分吃力。除此之外，因为模型是固定的，云海的运动没有办法模拟，也无法穿过云层产生交互。

体积云方法对云进行体素建模，在体素内定义云的体积场，在体积场内某一点的采样值定义为云在该点的密度。主要使用两种方法，一种是通过程序化的 3D 噪声生成体积场，一种是使用 3D 体纹理的方式生成体积场。其中 3D 体纹理可以在 Houdini 中比较方便的创建，并且可以方便艺术家控制云的效果。而 3D 噪声生成的体积场适合做大规模的云海。

2.2 基于物理的渲染技术概述

基于物理的渲染技术^[29]（Physically Based Rendering，简称 PBR）指的是使用基于物理原理和微平面理论来渲染阴影和光照模型，以及使用测量的表面参数来精确表示现实世界中的材料的方法理念。迪士尼在 SIGGRAPH 2012 上提出了著名的“迪士尼原则 BRDF（Disney Principled BRDF）”^[30]，由于其易用性和便捷的工作流程，它已被广泛用于电影和游戏行业。PBR 的核心理论包括：

微平面理论（Microfacet Theory）。微平面理论是将物体的表面建模为无数微观尺度上具有理想镜面反射的小平面（microfacet）的理论，每个小平面对具有随机方向。在实际的 PBR 工作流中，此类物体表面不规则性由粗糙度贴图或者高光度贴图来表示。

能量守恒（Energy Conservation）。出射光的能量永远不能超过入射光的能量。随着粗糙度的增加，镜面反射区域的面积增加。作为平衡，镜面反射区域的平均

亮度降低。

菲涅尔反射(Fresnel Reflectance)。以不同角度入射的光将具有不同的反射率。在相同的入射角下,不同的材料将具有不同的反射率,一切都有菲涅尔反射。 F_0 是以0度角入射的菲涅尔反射值。大多数非金属的 F_0 范围为0.02~0.04,大多数金属的 F_0 范围为0.7~1.0。

线性空间(Linear Space)。光照计算必须在线性空间内完成,需要将在着色器中输入的伽玛空间贴图(例如漫反射贴图)转换为线性空间,在具体操作时,需要根据不同的引擎和渲染器执行不同的操作。描述对象表面特征的纹理(例如粗糙度、高光贴图、金属贴图)必须保证是线性空间的。

色调映射(Tone Mapping)。也称为色调复制(tone reproduction),是使各种照明级别拟合到屏幕有限色域的过程。由于基于HDR渲染的亮度值会超过显示器可以显示的最大亮度,因此需要使用色调映射将光照结果从HDR转换为显示器可以正常显示的LDR。

物质的光学特性(Substance Optical Properties)。现实世界中存在不同类型的物质,可分为三类:绝缘体(Insulators),半导体(semi-conductors)和导体(conductors)。在渲染和游戏中,我们通常只对其中两种感兴趣:导体(金属)和绝缘体(电解质,非金属)。其中非金属具有单色/灰色镜面反射颜色。金属具有彩色镜面反射颜色。

当光在与非光学平坦表面(Non-Optically-Flat Surfaces)相互作用时,非光学平坦表面的表现就像一个微小光学平面表面的集合。表面上的每个点都以稍微不同的方向反射入射光,并且最终的表面外观是许多具有不同表面取向的点集合的结果。

在微观尺度上,表面越粗糙,反射越模糊,因为表面取向与整个宏观表面取向的偏离更大。从宏观角度来看,非光学平面可以看作是在多个方向上反射和折射光。

出于着色目的,我们通常使用统计方法来处理这种微观几何现象,并将表面视为在多个方向上每个点的反射(折射)光。

从表面折射的光是基于物体本身的特性的：对于金属，折射光会立即被吸收，即能量被自由电子立即吸收；对于非金属（也称为电介质或绝缘体），一旦它们中的光发生折射，常规的参与介质就表现出吸收和散射两种行为。

另外，漫反射和次表面散射实际上是相同的物理现象，并且本质上是折射光的次表面散射的结果。唯一的区别是相对于观察尺度的散射距离。与像素相比，散射距离可以忽略不计，次表面散射可以通过漫反射近似。换句话说，根据观察尺度，将光的折射现象建模为漫反射或次表面散射。

2.3 双向散射分布函数概述

双向散射分布函数^[31]（Bidirectional Scattering Distribution Function, 简称 BSDF）是研究物体表面粗糙度之光学性质，通常用在计算光在物体表面的散射。BSDF 由 Bartell, Dereniak 在 1980 年提出，它是一个统称，一般包括双向反射分布函数^[32]（Bidirectional Reflectance Distribution Function, 简称 BRDF）和双向透射分布函数（Bidirectional Transmittance Distribution Function, 简称 BTDF），反映了一个面上不同方向的散射强度。

由于物体表面上有凹凸不平的微小表面，一道入射光线射到表面而产生散射现象，用 BSDF 来表示这种散射现象。其中双向（Bidirectional）系指入射光与接受散射光的方向，因不同的入射光角度所产生的散射性质亦不相同。BSDF 主要应用在计算机仿真物体表面明亮度，与真实人眼所看物体之明亮度相符；另外可应用在背光模块之光学模拟，例如扩散膜之散射性质。

如图 2-1 所示，光线在经过一个物体面后，一般会分成 5 个部分，镜面反射、折射、BTDF、BRDF、还有吸收的部分。BRDF 和 BTDF 有时也称之为前向散射和后向散射。

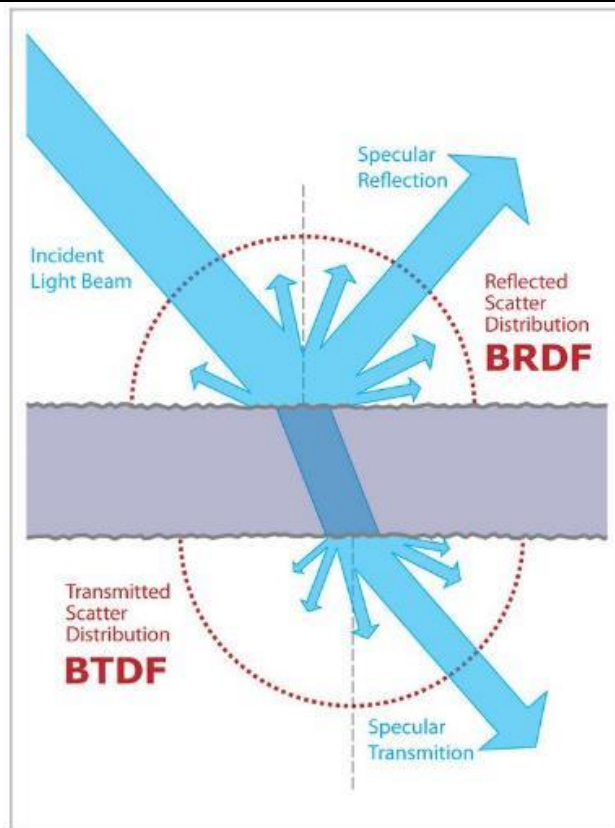


图 2-1 BSDF 的构成

(图片来自互联网 双向散射分布函数-百度百科)

渲染方程作为渲染领域中的重要理论，将 B_xDF 代入渲染方程是求解渲染问题的一般方法。其描述了光能在场景中的流动，是渲染中不可感知方面的最抽象的正式表示。根据光学的物理学原理，渲染方程在理论上给出了一个完美的结果，而各种各样的渲染技术，只是这个理想结果的一个近似。

渲染方程的物理基础是能量守恒定律。在一个特定的位置和方向，出射光 L_o 是自发光 L_e 与反射光线之和，反射光线本身是各个方向的入射光 L_i 之和乘以表面反射率及入射角。

这个方程经过交叉点将出射光线与入射光线联系在一起，它代表了场景中全部的光线传播路径。所有更加完善的算法都可以看作是这个方程的特殊形式的解。

某一点 p 的渲染方程，可以表示为：

$$L_o = L_e + \int_{\Omega} f_r \cdot L_i \cdot (w_i \cdot n) \cdot dw_i \quad (2-1)$$

其中 L_o 是 p 点的出射光亮度， L_e 是 p 点发出的光亮度， f_r 是 p 点入射方向到出射方向光的反射比例，即 BxDF，一般为 BRDF， L_i 是 p 点入射光亮度， $(w_i \cdot n)$ 是入射角带来的入射光衰减， $\int_{\Omega} f_r \cdot L_i \cdot (w_i \cdot n) \cdot dw_i$ 是入射方向半球的积分，即无穷小的累加和。

2.4 云的光照概述

如图 2-2 所示，光通量在进入穿过云层时会发生以下三种事件：外散射、吸收、内散射。

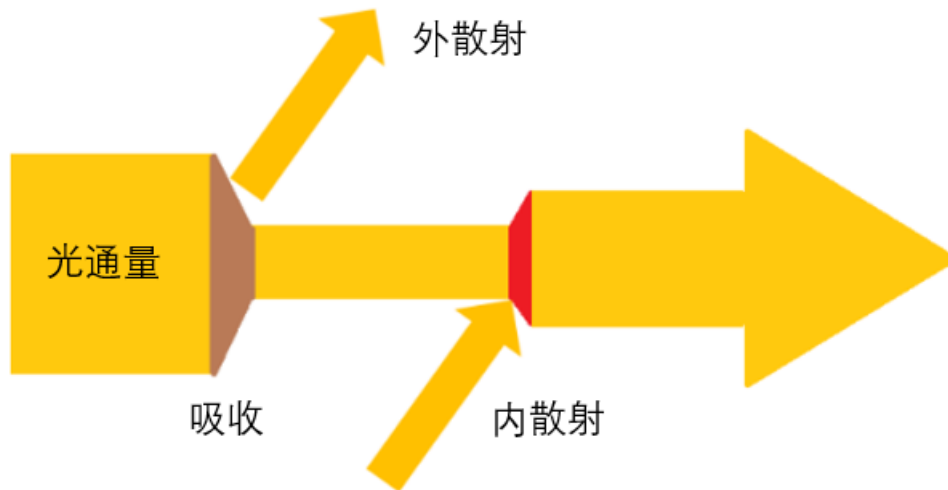


图 2-2 光通量的外散射、吸收和内散射

比尔-朗伯定律(Beer-Lambert law)表示物质对光吸收的定量关系，由皮埃尔·布格(Pierre Bouguer)和约翰·海因里希·朗伯(Johann Heinrich Lambert)分别在 1729 年和 1760 年阐明了物质对光的吸收程度和吸收介质厚度之间的关系；由奥古斯特·比尔(August Beer)在 1852 年又提出光的吸收程度和吸光物质浓度也具有类似关系，两者结合就得到光吸收的基本定律：比尔-朗伯定律。

比尔-朗伯定律是分光光度法的基本定律，是描述物质对某一波长光吸收的强弱与吸光物质的浓度及其液层厚度间的关系，是光吸收的基本定律，适用于所有的电磁辐射和所有的吸光物质，包括气体、固体、液体、分子、原子和离子。定律表示光被吸收的量正比于光程中产生光吸收的分子数目：

$$\tau(d) = e^{-\sigma_t d} \quad (2-2)$$

这表示光子延长度为 d 的路径穿过介质不碰到粒子的概率，也代表介质的透明度。从宏观上来讲光被透明介质吸收的比例与入射光的强度无关；在光程上每等厚层介质吸收相同比例值的光。

当通过分子和粒子撞击时会失去能量，但是也会有能量从其他地方经过撞击向视角方向过来。这会增加一些能量甚至可能弥补消光带来的能量损失。这种现象成为内散射。人们通过相位函数（The Phase Function）来估计光子从一个特定方向反弹到另一个方向的概率，特别是视角方向。相位函数和 BRDF 相似：通过一个黑盒模型来指示光线如何与材质交互，但是与 BRDF 不同的是，相位函数只需要考虑一个角度（2 个方向之间的相位角）并且在所有方向内积分为 1：

$$\int_{\Omega_{4\pi}} p(\omega_v, \omega_o) d\omega_o = 1 \quad \forall \omega_i \quad (2-3)$$

其中， $\theta = \widehat{\omega_i, \omega_o}$ 是进出方向之间的夹角， $\Omega_{4\pi}$ 代表球体所有的可能方向。

相位函数的平均余弦给出 g 用来定义光线倾向的散射方向，其取值范围在 -1 和 +1 之间。当 $g=-1$ 时，发生后向散射，光线将向光源方向反射，当 $g=1$ 时，发生前向散射，光线将继续前进不受影响：

$$g = \int_{\Omega_{4\pi}} p(\theta) \cos(\theta) d\omega_o \quad (2-4)$$

相位函数有着一些已经存在的分析模型。通常有 Rayleigh Phase Function 和 Henyey-Greenstein Phase Function 两种。

当粒子足够小时，Rayleigh 散射就会发生，如图 2-3 所示，其相位函数形状呈花生状。

$$p(\theta) = \frac{3}{16\pi} (1 + \cos(\theta)^2) \quad (2-5)$$

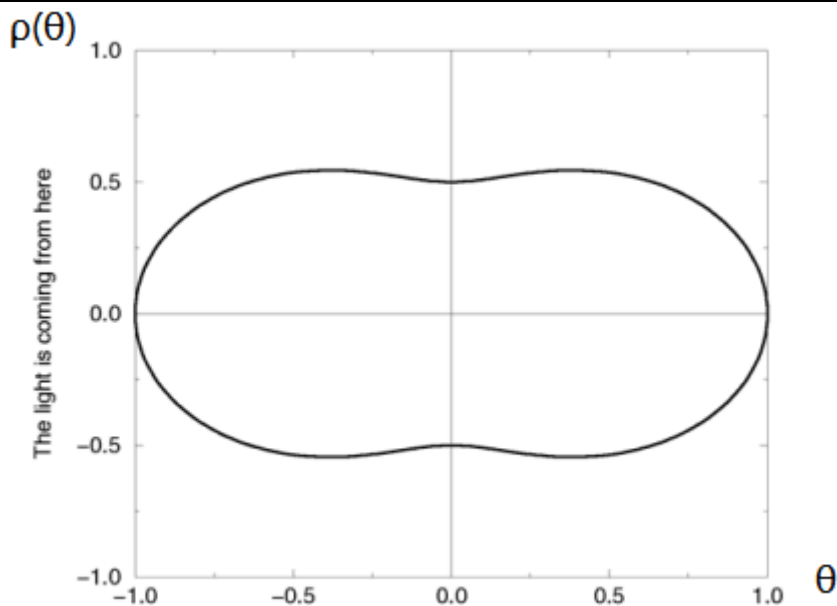


图 2-3 Rayleigh 相位函数（x 轴为相角，y 轴为对应的相位函数值）

（图片来自 SIGGRAPH 2015 Course Notes）

其中当分子的大小和光的波长是相同数量级时，分子散射的重要性与波长的四次方成反比：

$$I_{\text{scattered}} \propto \frac{1}{\lambda^4} \quad (2-6)$$

这意味着 400nm 处的散射约为 700nm 处的 9 倍。这也是天空是蓝色的主要原因：较短的波长（蓝色）比较长的波长（红色）更容易受到干扰。

对于大一点的粒子，如污染物、灰尘和水滴等，这个时候会发生米氏散射^[33]（Mie scattering）。米氏散射是当大气中粒子的直径与辐射的波长相当时发生的散射，这种散射主要由大气中的微粒，如烟、尘埃、小水滴及气溶胶等引起。米氏散射的散射强度与波长的二次方成反比，并且散射在光线向前方向比向后方向更强，方向性比较明显。

云的平均统计相位函数如图 2-4 所示，因为大气中各个组成部分的形状和行为的多样性和复杂性，使得基于米氏散射理论的相位函数难以使用。

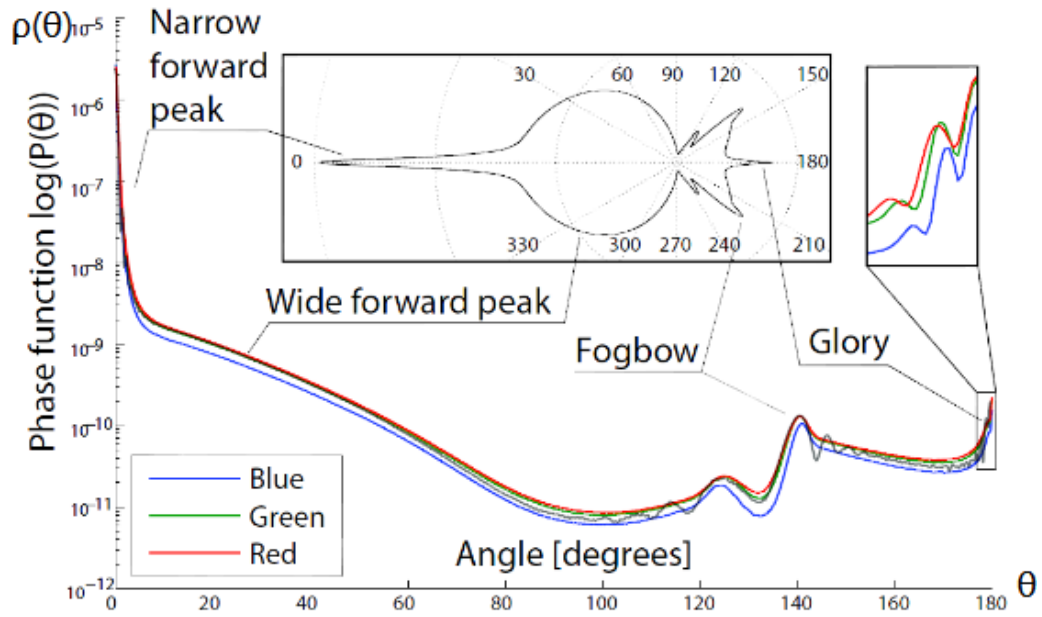


图 2-4 云的平均统计相位函数（x 轴为相角，y 轴为对应的相位函数值）

（图片来自 SIGGRAPH 2015 Course Notes）

人们通常使用 Henyey-Greenstein 相位函数^[34]（Henyey-Greenstein phase function，简称 HG 相位函数）来代替使用复杂的相位函数，该相位函数由多个简化函数组合而成：

$$p_{HG}(\theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g \cos(\theta))^{3/2}} \quad (2-7)^{[34]}$$

其中 g 和之前的 g 一样，为倾向散射方向参数，取值范围为 $[-1, +1]$ 。

HG 相位函数形状如图 2-5 所示。

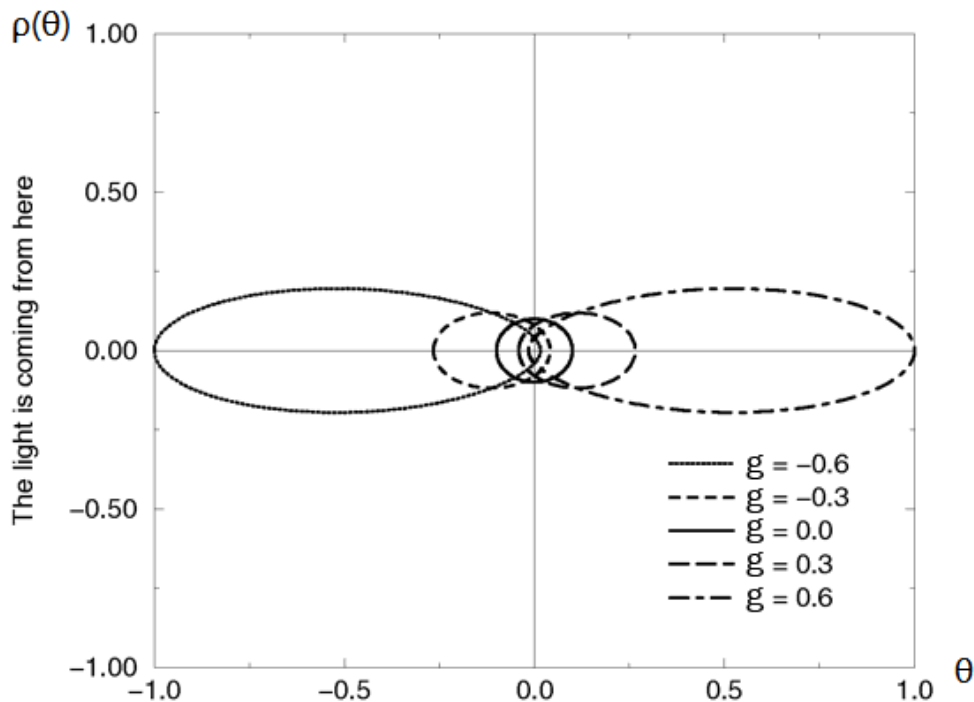


图 2-5 HG 相位函数 (x 轴为相角, y 轴为对应的相位函数值)

(图片来自 SIGGRAPH 2015 Course Notes)

相位函数可以使用权重累计计算, 假设权重之和为 1:

$$p(\theta) = w_0 \cdot p_{HG}(\theta) + w_1 \cdot p_{HG}(\theta) + (...) + w_i \cdot p_{HG}(\theta) \quad \sum w_i = 1 \quad (2-8)$$

因此, 我们可以使用简化相位函数的权重和来表示复杂的米氏散射。

2.5 高阶着色器语言概述

着色器语言 (Shader Language, 简称 SL) 是一类专门用来为着色器编程的编程语言。这类语言使用“颜色”、“法线”等特殊的数据类型。由于三维计算机图形目标市场的多元化, 不同的目标市场通常使用不同的着色器语言。着色器语言在实时计算机图形领域有着广泛应用。与此前常见的硬编码方式相比, 这类语言对于硬件抽象的程度很高, 同时还给予了程序员更大的灵活性, 可以更好地控制

整个渲染过程。由于串流处理的特点，这类直接在 GPU 上运行的着色器也常被用于吞吐量较大的通用数据处理领域。

Shader Language 的发展方向是设计出在便携性方面可以和 C++、Java 等相比的高级语言，“赋予程序员灵活而方便的编程方式”，并“尽可能的控制渲染过程”同时“利用图形硬件的并行性，提高算法效率”。Shader Language 目前主要有 3 种语言：基于 OpenGL 的 OpenGL 着色器语言（OpenGL Shading Language，简称 GLSL）；基于 DirectX 的高阶着色器语言（High Level Shading Language，简称 HLSL）；还有 NVIDIA 公司的 Cg 语言（C for Graphic，简称 Cg）。

其中 HLSL 是 DirectX 的高级着色语言，使用 HLSL 可以为 Direct3D 管道创建类似于 C 的可编程着色器。HLSL 从 DirectX 9 开始诞生，用以设置可编程的 3D 管道，可以使用 HLSL 指令对整个管道进行编程。HLSL 是 DirectX 的高级着色语言，使用 HLSL 可以为 Direct3D 管道创建类似于 C 的可编程着色器。HLSL 的主要作用为将一些复杂的图像处理，快速而又有效率地在显示卡上完成，与组合式或低阶着色器语言相比，能降低在编写复杂特殊效果时所发生编程错误的机会。

2.6 虚幻 4 引擎概述

虚幻引擎 4^[35]（Unreal Engine4，简称 UE4）是 Epic Games 公司开发的第四代游戏引擎，首次亮相是 1998 年的第一人称射击游戏“Unreal”。当时主要为第一人称动作类游戏开发，如今已成功用于各种其他类型，包括格斗游戏、MMORPG 和其他 RPG 游戏。在过去的二十多年间，Epic 已经使用自研的虚幻引擎制作出了众多备受赞誉的内容，并由此开发出了强大的工具和高效的制作流程，虚幻引擎已经成为一个稳定可靠的引擎。UE4 使用 C++编写代码也可以使用可视化编程的蓝图系统，同时支持众多的开发平台如 Windows、IOS、Android、HTML5 及几乎所有热门的虚拟现实设备和游戏平台，具有高度的可移植性，是当今众多游戏开发人员使用的工具。

UE4 是一套完整的创新、设计工具，具备足够的灵活性，可满足不同的开发

需求。作为一个成熟的引擎，虚幻引擎功能强大，其预设的模块化系统、自定义插件以及源控制集成特性，能满足不同项目的独特需要。同时 UE4 的源码在 GitHub 上完全开源，故本文使用虚幻 4 引擎来进行云渲染相关技术在 Windows 平台以及 Android 平台的研究。

UE4 中包含许多模块，包括蓝图系统、粒子系统、动画系统、材质系统、光照系统、着色模型等等。

材质系统中使用高阶着色器语言（High Level Shader Language，简称 HLSL）。通过材质编辑器中的可视化脚本节点（称为材质表达式）所组成的网络来构建。每一个节点都包含 HLSL 代码片段，并用于执行特定的任务。这意味着构建材质时，可以通过可视化脚本编程来创建 HLSL 代码，也可以通过自定义材质表达式来添加自己编写的 HLSL 代码。通常情况下颜色在数字成像方面，由 RGBA4 个主通道构成，对所有数字图像中的每一个像素而言，其中任何通道的值都可以由一个数字表示。关于材质的许多工作无非是根据一系列的情况和数学表达式来处理这些数字。

着色模型使用基于物理的渲染技术（Physically Based Rendering，简称 PBR），UE4 的渲染系统使用 DirectX 11 管线，同时所有的光照均为延迟光照（Deferred），与前置光照（Forward）不同。材质将它们的属性写入 GBuffers，光照过程则读取材质每个像素的属性，并对他们执行光照处理。

本文主要使用 UE4 的材质系统中的自定义材质表达式以及修改 UE4 源码添加新的着色模型来实现云渲染相关研究内容。

2.7 本章小结

本章主要介绍云渲染相关技术。本章分别对基于物理的渲染技术、双向散射分布函数、高阶着色器语言、虚幻 4 引擎进行概述。论文中使用基于物理的渲染技术的思想和具体的双向散射分布函数对体积云进行渲染，使用高阶着色器语言对着色器进行编程，整个体积云的实现和优化基于虚幻 4 引擎，本章对以上技术进行了介绍。

第3章 体积云建模

本章提出一种基于噪声生成的体积云建模的方法，在完成体积云建模的基础上，根据云的特征添加更多细节。体积云方法是对云进行体素建模，即在体素内定义云的体积场，体积场内某一点的采样值定义为云在该点的密度，该方法是目前公认效果逼真的云的实时模拟方法。而通过噪声生成体积场来对体积云进行模拟，其结果具有随机性，更能表现出云的特征。Perlin 噪声被大量用于云朵的模拟，我们首先生成 Perlin 噪声，利用 Perlin 噪声产生云的雏形，此时噪声十分粗糙，缺少细节。为了使云的建模更加逼真，更接近现实生活中自然形态的云，我们接下来通过分形噪声将 Perlin 噪声叠加来为其增加细节，通过分形噪声可以把多个不同振幅、不同频率的噪声相叠加，得到一个更加自然的噪声。但是我们发现只通过 Perlin 噪声无法表现出云的翻涌特征，而 Worley 噪声经常被用于火焰的模拟，能够表现出火势汹涌的特征，经实验发现当 Perlin 噪声与 Worley 噪声以恰当的方式结合能够模拟出具有逼真的翻涌特征的云。又由于气压和气温随着高度的变化而改变，对天空中不同高度的云的密度也会有所影响，我们使用密度-高度函数来控制不同高度的云的密度，表现出不同高度的云的特征。又由于风和大气运动的缘故，云不是完全静止的，我们使用时间函数来控制云的运动，表现出云的运动特征。最终通过 Perlin 噪声与 Worley 噪声的分形噪声相结合作为噪声生成体积场，加入密度-高度函数和时间函数，实现体积云的建模。

体积云方法主要使用两种方法，一种是使用 3D 体纹理的方式生成体积场，一种是通过程序化的 3D 噪声生成体积场。

3.1 基于体纹理的体积云生成

使用 3D 体纹理生成体积场来对体积云进行模拟，其 3D 纹理可以在 Houdini 中通过成熟方法创建，这种方法可以更方便的控制云的效果。不同于我们常用的储存在 2D 空间中的纹理，3D 体纹理将信息存储在 3D 空间。虽然图形 API 已经

支持了一段时间的体纹理，但并非所有引擎都支持 3D 体纹理，所以通常要使用常规 2D 纹理创建和使用“伪体纹理”。它们不会像真正的硬件支持的体积纹理那样节省系统开销，但是可以通过这种方式进行原型设计。

伪体纹理是一系列按顺序排列的 2D 纹理的集合。对于体积纹理，我们将另一个空间维度视为第三维，通常为 Z 轴。如图 3-1 所示，一个 16x16x16 的体纹理的可视化，它存储为一系列的 2D 纹理帧，左侧是所有帧垂直堆叠，代表每帧的编码位置，右侧是伪 3D 纹理的 2D 框架布局。每一帧存储有关 2D 空间切片的信息。所有帧都存储相同的 X, Y 位置，并且每帧都只会改变 Z 的位置。它共有 16 帧，与体纹理中的 Z 轴的大小匹配。理想情况下，每帧的分辨率也应为 16x16，以便在各个轴上保持相同的分辨率。

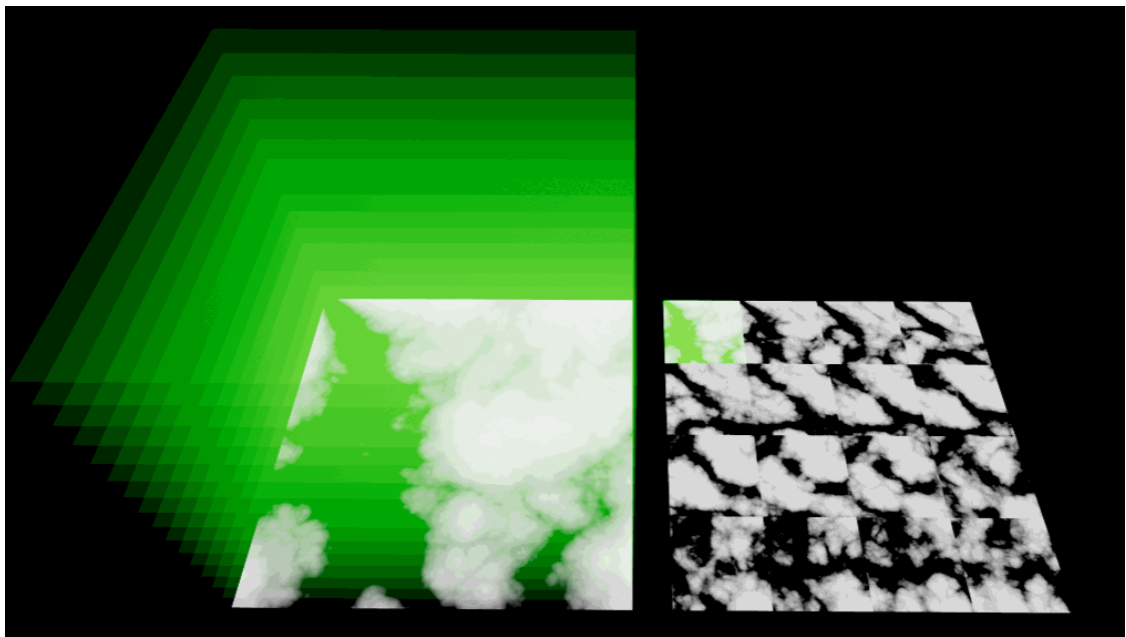


图 3-1 伪体纹理的构建

（图片来自互联网 Begin ray marching in unreal engine 4【第五卷：3D 体纹理云】-知乎）

要采样一张贴图，在 2D 情况下使用 UV 坐标，在 3D 体纹理下自然需要一个 3D 的参数来采样一个 3D 贴图，即 UVW。因为体纹理是一个方块，所以用 3D 单

位 Cube 来模拟他的 UVW。获取物体的 BoundingBox 然后把它归一成一个单位 Cube，然后根据 UVW 进行采样。需要注意的是，实际采样时需要对纹理进行两次采样才能在 Z 帧之间进行融合。这也是伪体纹理比真正硬件实现的体纹理要慢一些的原因。真正的体纹理使用硬件 mip 映射自动进行混合，而伪体纹理则需要 2 次采样进行插值手动混合。

3D 体纹理一般由艺术家制作，过程比较复杂，在这里我们通过简化流程生成一个简单的体纹理用来实验使用：先生成一个简单的球形模型，然后对于每个顶点进行随机形变，然后根据云的形状需要进行空间矩阵变换，然后在模型中添加噪声，再将模型转化为 vdb 格式，按 Z 轴对模型进行切片，存为 2D 纹理贴图集合，然后将 2D 贴图集合制作成为伪体纹理贴图，完成基于体纹理的体积云建模。

3.2 基于 3D 噪声的体积云生成

相比于使用 3D 体纹理生成体积场，通过噪声生成体积场来对体积云进行模拟，其结果具有随机性，更能表现出云的特征，并且更适合大规模云海的模拟。噪声是程序生成图形中最基础的工具，任何一种计算机图形工具都会用到噪声，在图形学中，噪声的使用是为了把一些随机变量来引入到程序中。通过不同的算法产生不同的变化，从而达到不同的效果。

噪声的基础来自于随机数，其中白噪声（White noise）是最基础的噪声，如图 3-2 所示，它是指功率谱密度在整个频域内均匀分布的噪声，即每个像素点的亮度用 0-1 之间的随机数生成，因为白噪声中每个点的随机数都是离散的，互相之间没有关系，而现实生活中自然界产生的噪声大多是有规律的，所以一般利用噪声来对自然景观进行建模时几乎不会直接使用白噪声。

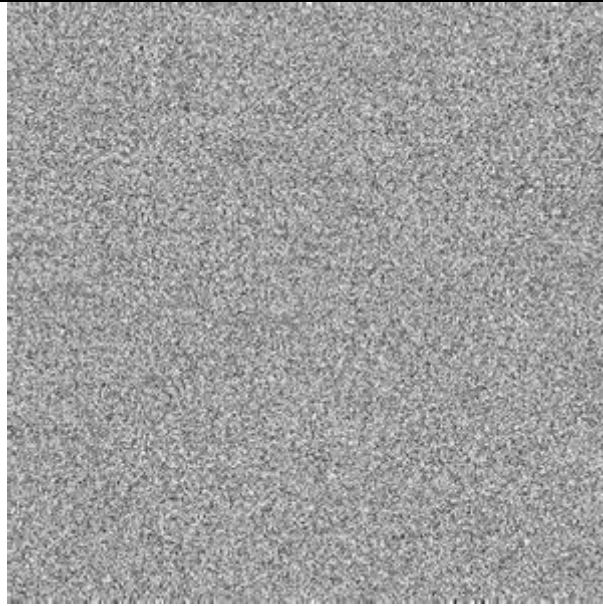


图 3-2 白噪声

白噪声与自然界中的噪声不同，嘈杂且没有规律，而现实生活中的一些噪声通常有规律可循，例如木头纹理、山脉起伏等，它们的虽然也是由噪声构成，但是其形状大多是趋于分形的，包含了不同程度的细节。其中地形中有起伏较大的山脉，有起伏较小的山丘，有细节非常多的石子，这种由不同程度的细节所构成的噪声叠加在一起，构成了现实生活中自然的地形表面。根据不同的需求人们提出了不同的由程序模拟自然噪声的方法，其中很多方法在图形学中有着广泛的应用：Perlin 噪声被大量用于云朵、火焰和地形等自然环境的模拟；Simplex 噪声与 Perlin 噪声类似，在其基础上进行了改进，构建方法从基于 N 维空间立方体改为基于 N 维空间单形，从而提高了效率和效果；Worley 噪声被提出用于模拟一些如纸张、木纹等的多孔结构。

Perlin 噪声被大量用于云朵的模拟，我们首先生成 Perlin 噪声，构建出体积云的雏形。

3.2.1 基于 Perlin 噪声的雏形构建

Perlin 噪声^{[8][36][37]}的名字来源于它的创始人 Ken Perlin。Perlin 噪声在所有整形顶点处的参数值为 0，并且变化来自顶点之间的梯度向量，然后进行平滑插值得到。由于 Perlin 噪声的算法简单，被迅速应用到各种商业软件中。

如图 3-3 所示，我们通过以下步骤生成 Perlin 噪声：

1、定义一个晶格结构，每个晶格的顶点生成一个“伪随机”的梯度向量。对于二维的 Perlin 噪声来说，晶格结构就是一个正方形网格，三维的就是一个立方体网格。

2、输入一个点，找到和它相邻的那些晶格顶点（二维下有 4 个，三维下有 8 个， n 维下有 $2n \times 2n$ 个），计算该点到各个晶格顶点的距离向量，再分别与顶点上的梯度向量做点乘，得到 $2n \times 2n$ 个点乘结果。

3、使用缓和曲线（ease curves）来计算它们的权重和。原始的 Perlin 噪声实现中，缓和曲线是 $s(t) = 3t^2 - 2t^3$ ，在 2002 年的论文中 Perlin 改进为 $s(t) = 6t^5 - 15t^4 + 10t^3$ 。如果直接使用 $s(t) = t$ ，即线性插值，它的一阶导在晶格顶点处（即 $t = 0$ 或 $t = 1$ ）不为 0，会造成明显的不连续性。而 $s(t) = 3t^2 - 2t^3$ 在一阶导满足连续性， $s(t) = 6t^5 - 15t^4 + 10t^3$ 在二阶导上仍然满足连续性。

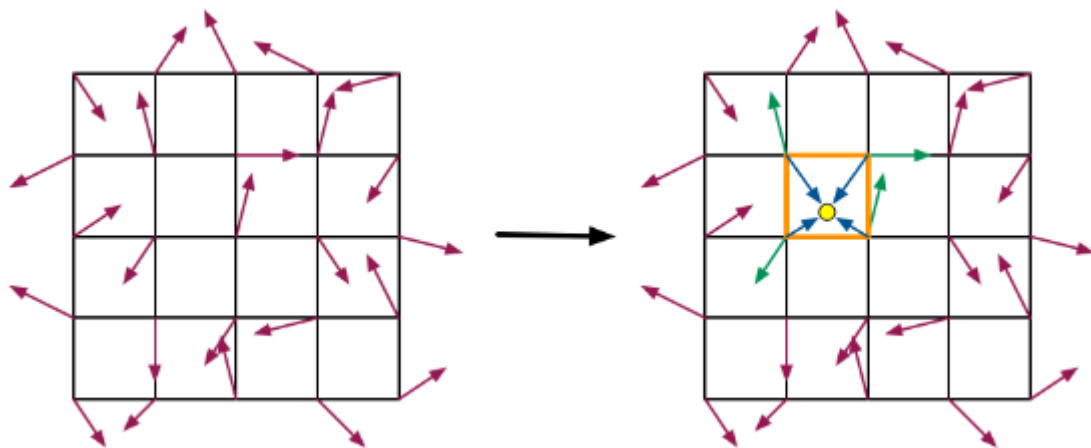


图 3-3 Perlin 噪声生成方法

（图片来自于互联网 【图形学】谈谈噪声-CSDN 博客）

这里我们首先利用 Perlin 噪声产生了云的雏形，将其进行适当放大，如图 3-4 所示，此时的噪声仍然十分粗糙，缺少细节。

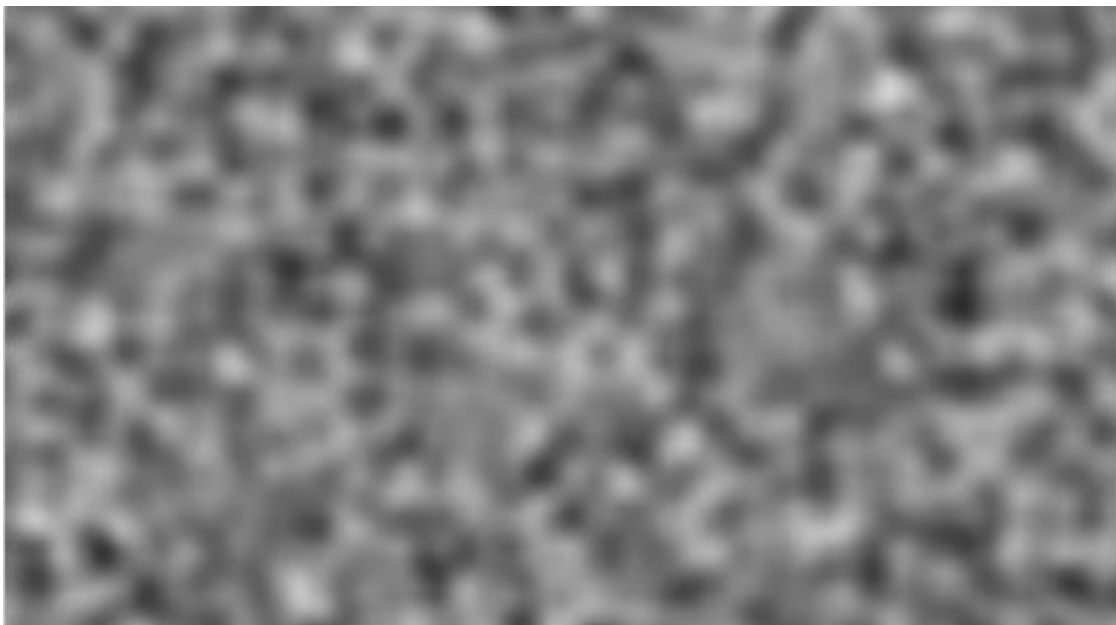


图 3-4 放大 8 倍的 Perlin 噪声

为了使云的建模更加逼真，更接近现实生活中自然形态的云，我们接下来通过分形噪声将 Perlin 噪声叠加来为其增加细节。

3.2.2 基于分形噪声的细节构建

不同噪声之间可以相互叠加，称为分形噪声（Fractal noise）。分形噪声会把多个不同振幅、不同频率的倍频（Octave）相叠加，得到一个更加自然的噪声。而这些倍频则对应了不同的来源，它可以是 Perlin 噪声，也可以是一个简单的白噪声。分形布朗运动（Fractal Brownian Motion，简称 fBm）是一种常用的生成噪声图的方法，其本身的数学特性可以生成许多独特视觉效果的图形。由于它本身的分形特性，通过 fBm 可以生成具有丰富细节的噪声。fBm 就是将很多个不同频率、不同振幅的基础噪声之间相互叠加，最后形成了最终的分形噪声的方法。其

叠加公式为：

$$\sum_{i=0}^n \frac{\text{Noise}(2^i \text{point})}{2^i} \quad (3-1)$$

这里我们利用 fBm 将 5 个不同 Octave 的 Perlin 噪声叠加，为其增加更多细节。

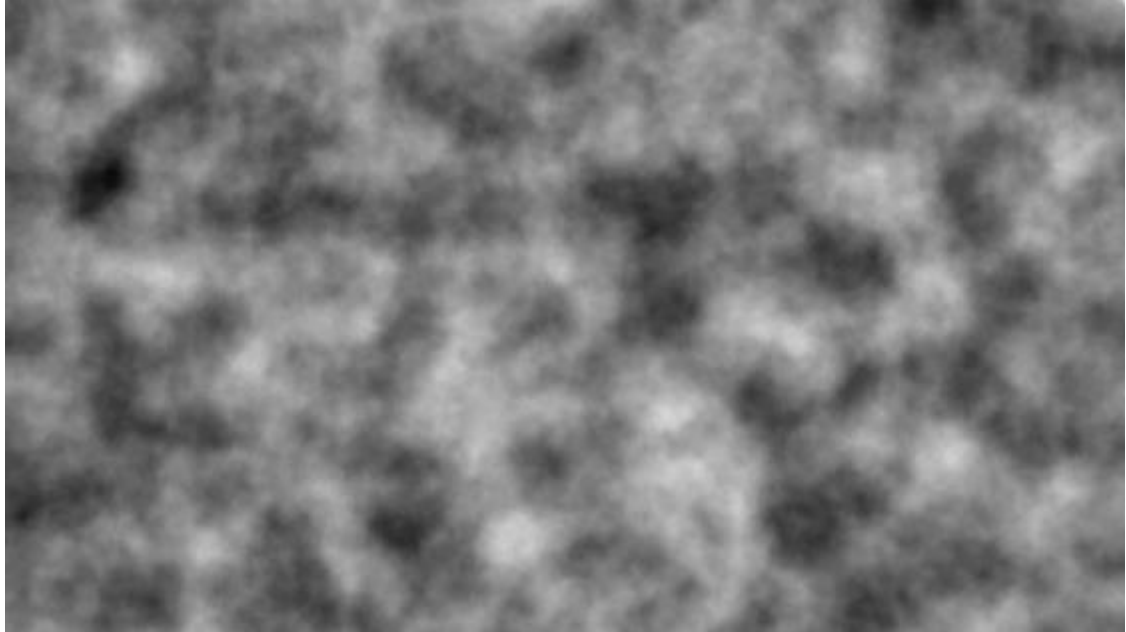


图 3-5 5 个不同 Octave 的 Perlin 噪声叠加

如图 3-5 所示，可以看到此时已经有了云的初步形状，但是仍然比较模糊，缺乏运动感，没有云层翻涌的视觉效果。我们在此基础上对 Perlin 噪声取绝对值，锐化 0 值区域附近的过度和变化。

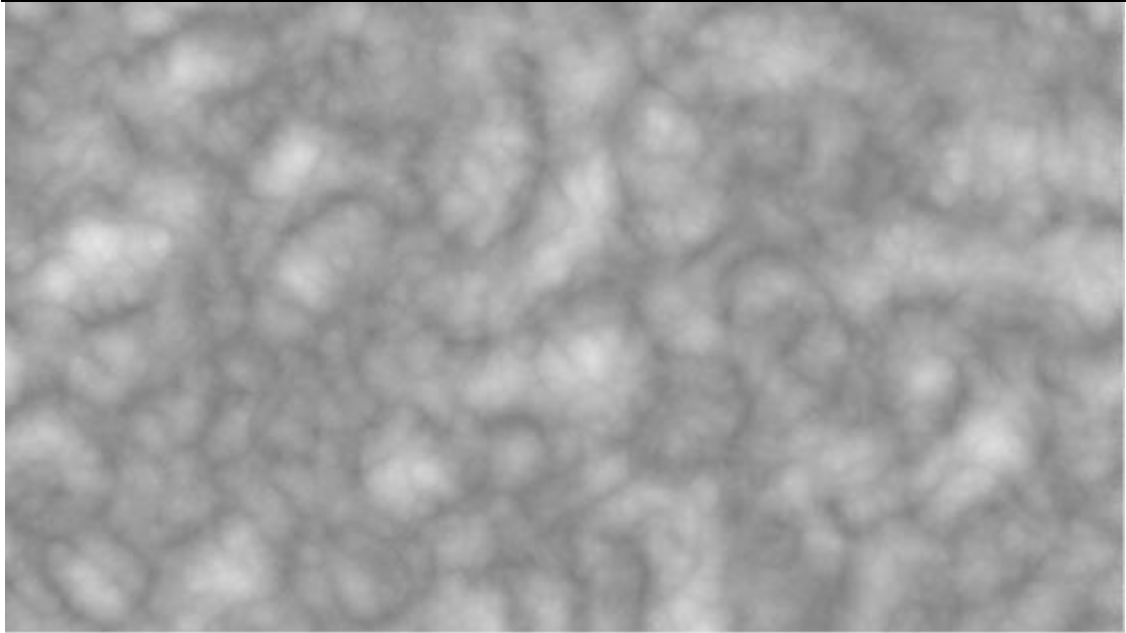


图 3-6 5 个不同 Octave 的 Perlin 噪声的绝对值叠加

因为使用绝对值操作，在 0 值区域附近的变化会出现不连续性，产生锐化效果。如图 3-6 所示，可以看到此时已经有了比较好的效果，但是这些噪声几乎连成一片，而自然界中的云会一朵一朵在天空中呈现离散状。因此我们在此基础上再进行一系列变换操作，使其更加蓬松，结果如图 3-7 所示。

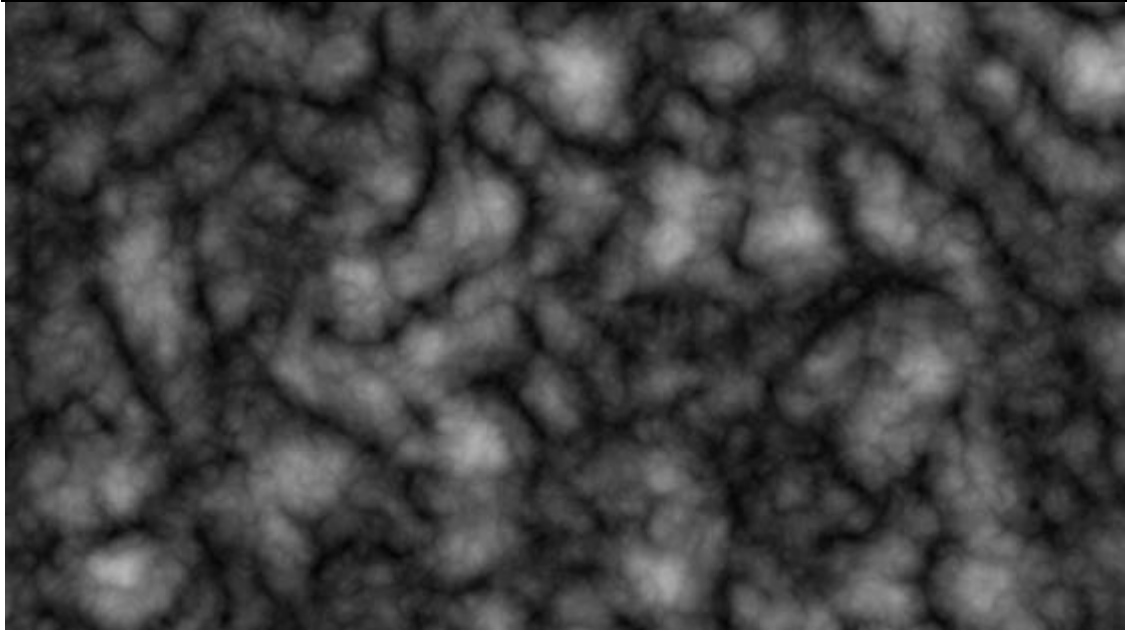


图 3-7 5 个不同 Octave 的 Perlin 噪声的绝对值变换叠加

但是单纯的 Perlin 噪声仍然无法得到如图 3-8 所示现实中的云效果，有花朵或浪花一样的视觉效果。



图 3-8 现实中自然景观的云

我们发现只通过 Perlin 噪声无法表现出云的翻涌特征，而 Worley 噪声经常被用于火焰的模拟，能够表现出火势汹涌的特征，我们接下来引入新的 Worley 噪声，将其与 Perlin 噪声结合，模拟出具有逼真的翻涌特征的云。

3.2.3 基于 Worley 噪声的低频细节构建

Worley 噪声常用来模拟细胞类有孔纹理，是一种基于 Voronoi 图的噪声生成算法，1996 年，在 Siggraph96 上，Steven Worley^[38]提出了一种用于实现 cellular texture 方法，能在有限的资源时间内快速生成噪声。

我们通过以下步骤生成 Worley 噪声：

- 1、确定输入点所在的晶胞。
- 2、对该晶胞生成一个随机数生成器，该随机数生成器可重复使用，通过随机数生成器生成多组特征点。

- 3、确定在该晶胞内生成特征点的数量。
- 4、随机将3组生成的特征点放置到晶胞中。
- 5、计算输入点到该晶胞内所有特征点的距离的最小值。
- 6、查找并计算输入点到所在晶胞直接相邻晶胞内特征点的距离最小值，并与5中生成的最小值比较，返回最小值。

如图3-9所示，目前生成的Worley噪声还看不出端倪。

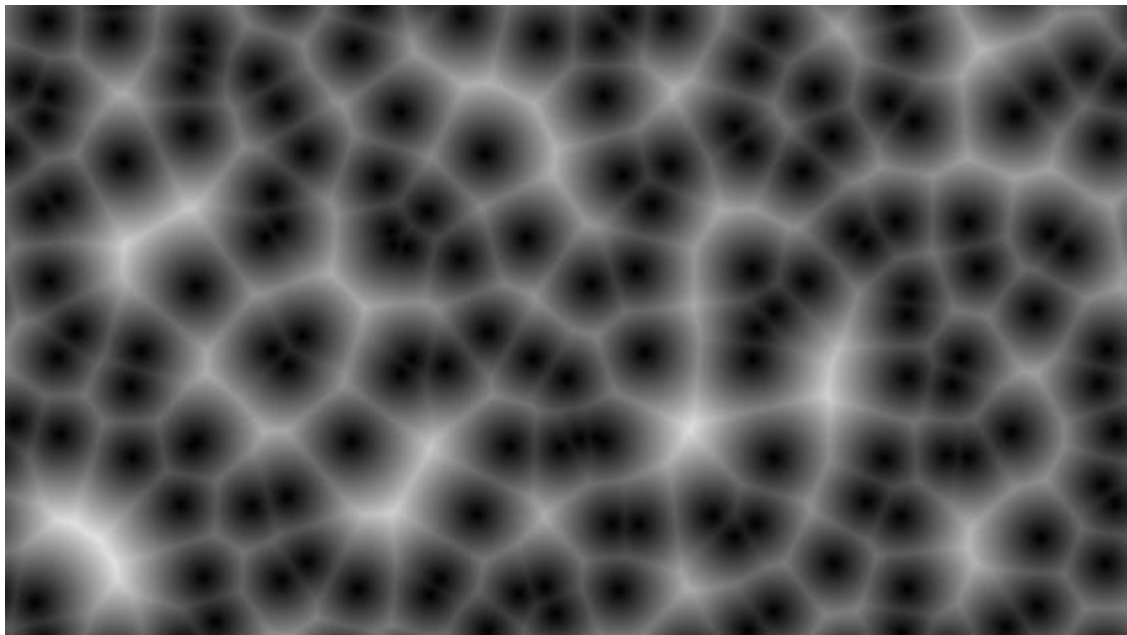


图 3-9 Worley 噪声

接下来我们把Worley噪声取反，如图3-10所示，得到了一个很好的分形翻滚的图案，并将它用于为低频Perlin噪声的低密度区域添加细节。

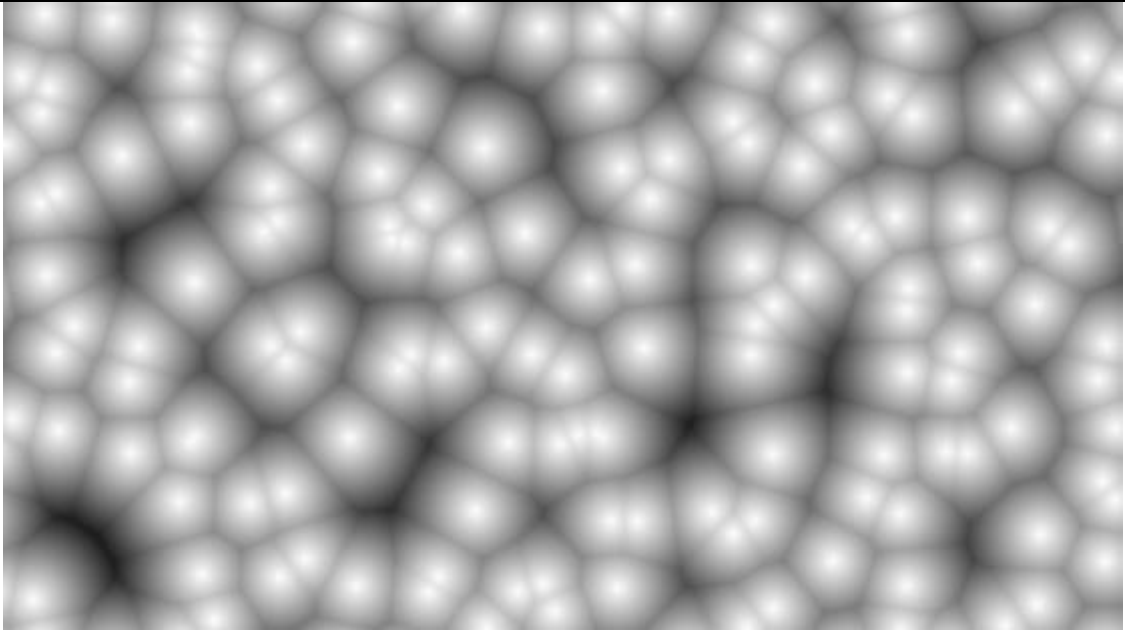


图 3-10 Worley 噪声取反

同样，这里我们利用 fBm 将 5 个不同 Octave 的 Worley 噪声叠加，为其增加更多细节，结果如图 3-11 所示。

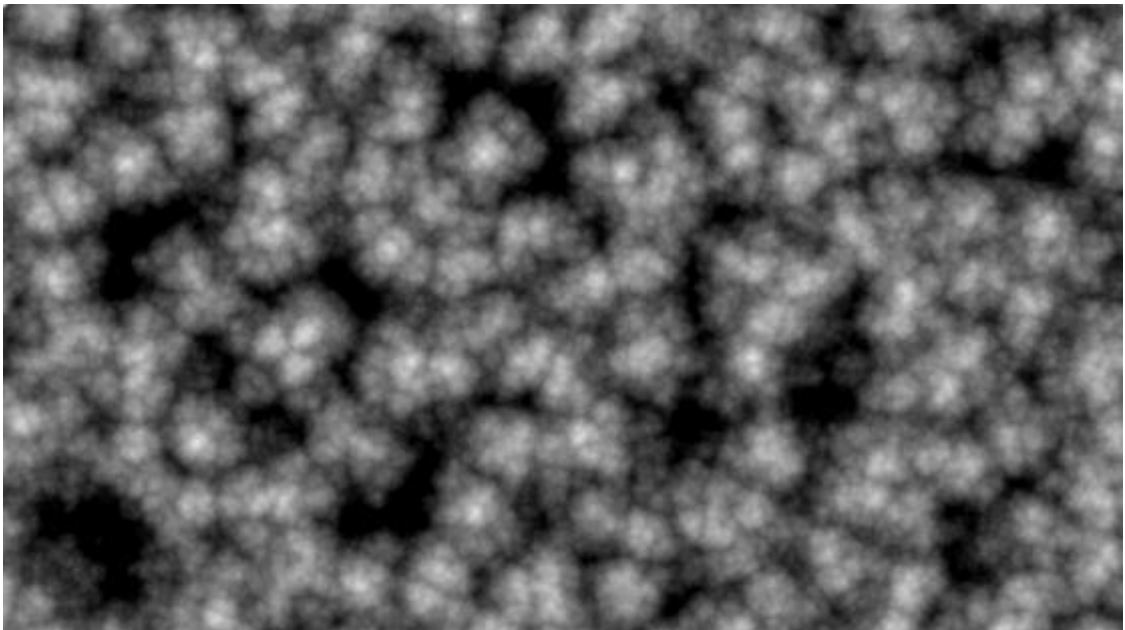


图 3-11 5 个不同 Octave 的 Worley 噪声叠加并取反

将 Perlin 噪声和 Worley 噪声这两种噪声相结合，为我们之前的噪声添加一些翻滚的效果，更接近于自然的云景。我们在这里使用重映射函数将经过 fBm 的 Perlin 噪声和 Worley 噪声叠加，具体的叠加方式为将 Worley 噪声作为 Perlin 噪声原始范围的最小值后进行归一化操作，结果如图 3-12 所示。

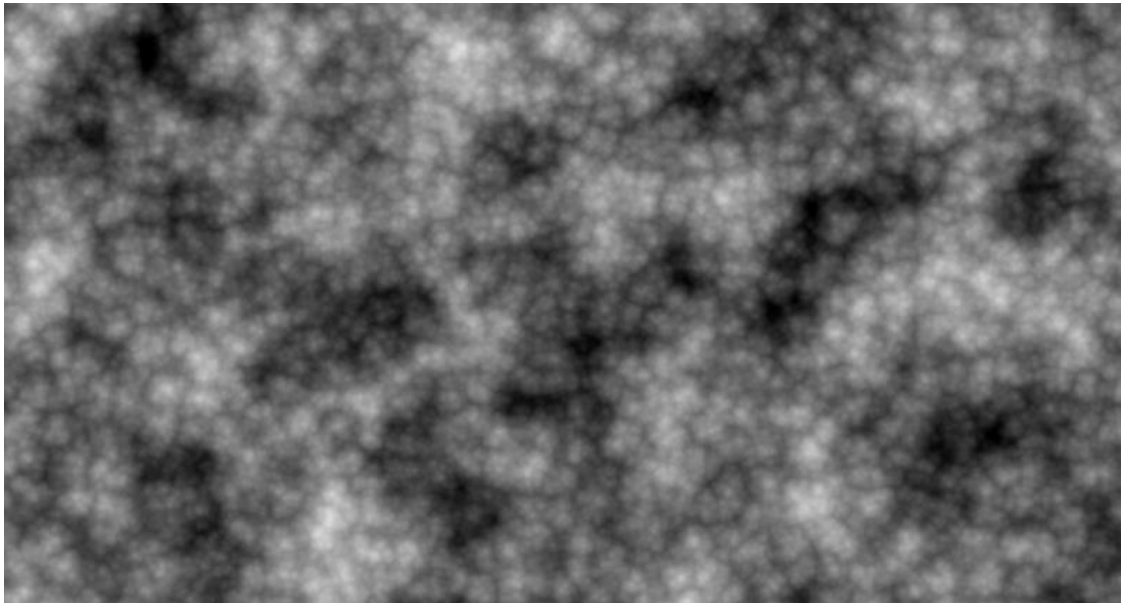


图 3-12 Perlin 噪声在 Worley 噪声上重映射

最终我们通过 Perlin 噪声与 Worley 噪声的分形噪声相结合（简称 Perlin-Worley 噪声），初步完成云的体积场的构建。接下来继续为云的体积场添加一些云的特征。

3.2.4 密度-高度函数

由于气压和气温随着高度的变化而改变，对天空中云的密度也会有所影响，即天空中不同高度的云的密度不是一成不变的。不同高度也存在着不同类型的云。在 2000 米以下的天空中，其主要构成的云的类型有 3 类：层积云 (Stratocumulus)、积云 (Cumulus) 和层云 (Stratus)。其中层积云在三种云中最高，其云块较大，

云的底部有着清晰的轮廓；而积云稍低，其顶部凸起，底部几乎水平，是最常见的云的类型，云的边界清晰；层云最低，顾名思义其云体是层状的，整体呈灰色，与雾相似。

我们通过使用密度-高度函数来控制不同高度的云的密度，对噪声进行缩放和偏移操作，从而达到控制云在不同高度有着不同密度的目的。同时也可以通过不同的密度-高度函数来控制不同类型的云的生成，从而实现不同类型云在不同高度上不同程度的混合。

3.2.5 时间函数

因为风和大气运动的缘故，云不是完全静止的，他会随着风一起运动。在这里我们定义时间函数来控制云的运动。其中定义两个参数：风速 v 和风向 dir ，让噪声随着时间的变化而进行相应的偏移。

最终通过 Perlin-Worley 噪声，密度-高度函数以及时间函数完成对于体积云体积场的构造。体积云体积场的最终函数表达式为：

$$D(p, t) = \frac{[fBm_5(abs(Perlin(p + v * dir * t))) - fBm_5(Worley(p + v * dir * t))]}{[1 - fBm_5(Worley(p + v * dir * t))]} \quad (3-2)$$

$$* HeightGradient(((p + v * dir * t).z))$$

其中 $D(p, t)$ 代表体积云体积场中某一点的密度， p 为点坐标， t 为时间。

3.3 实验结果

本章在实现体积云建模的基础上，通过将生成的 Perlin 噪声在 Worley 噪声上做重映射，实现细节更丰富的建模，噪声结果如图 3-13 所示。对比陈小龙使用 Perlin 噪声的方法^[23]以及 Schneider 使用 Perlin 噪声与 Worley 噪声线性混合的方法^[16]，单纯使用 Perlin 噪声在低密度区域几乎为空，与云的特征不符，使用 Perlin 噪声与 Worley 噪声混合则破坏了原本最符合云的特征的 Perlin 噪声的形态。噪声

结果如图 3-14,3-15 所示。对比可见本文噪声结果在保留了原 Perlin 噪声的整体形态基础上,有效的在低频 Perlin 噪声的低密度区域通过 Worley 噪声的重映射添加了更多的细节,实现了更加体现体积云细节的建模。

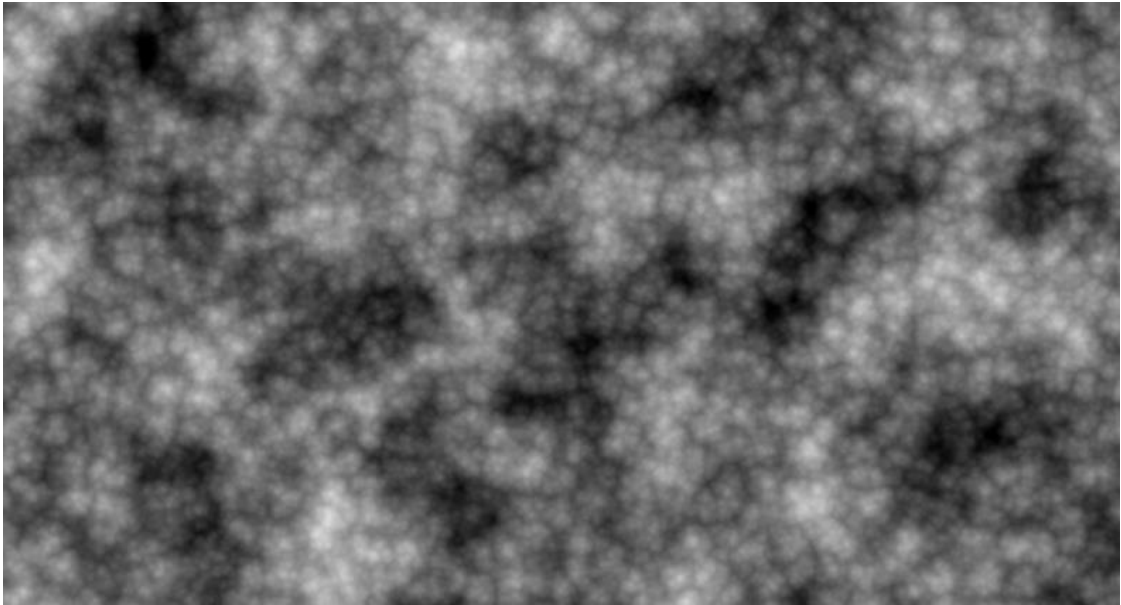


图 3-13 本文噪声结果

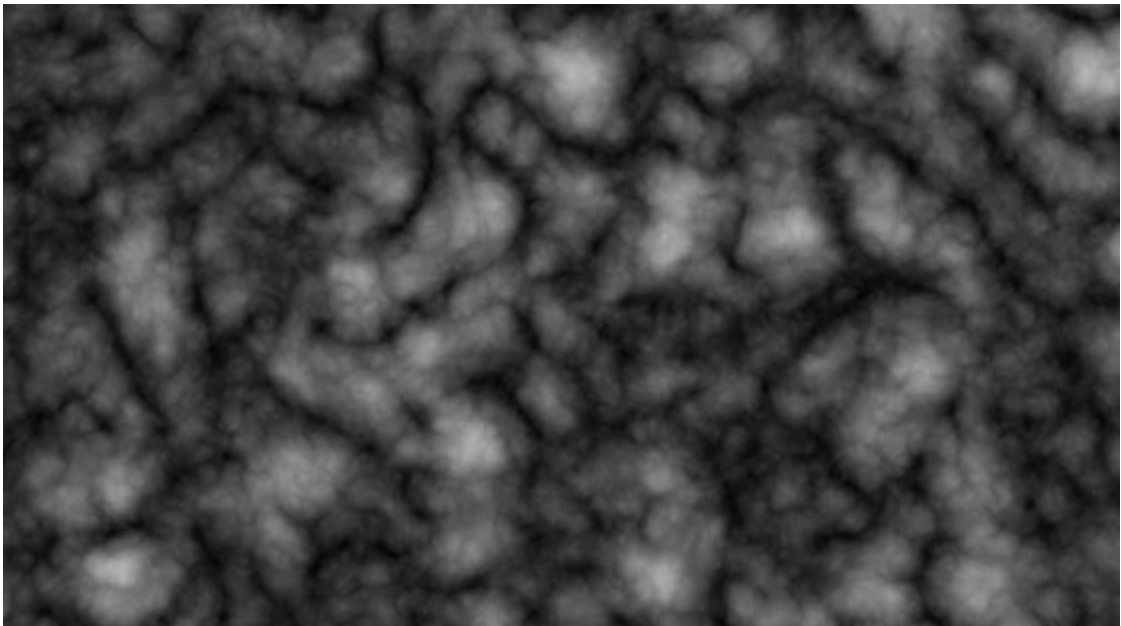


图 3-14 陈小龙 计算机技术与发展 2015 噪声结果

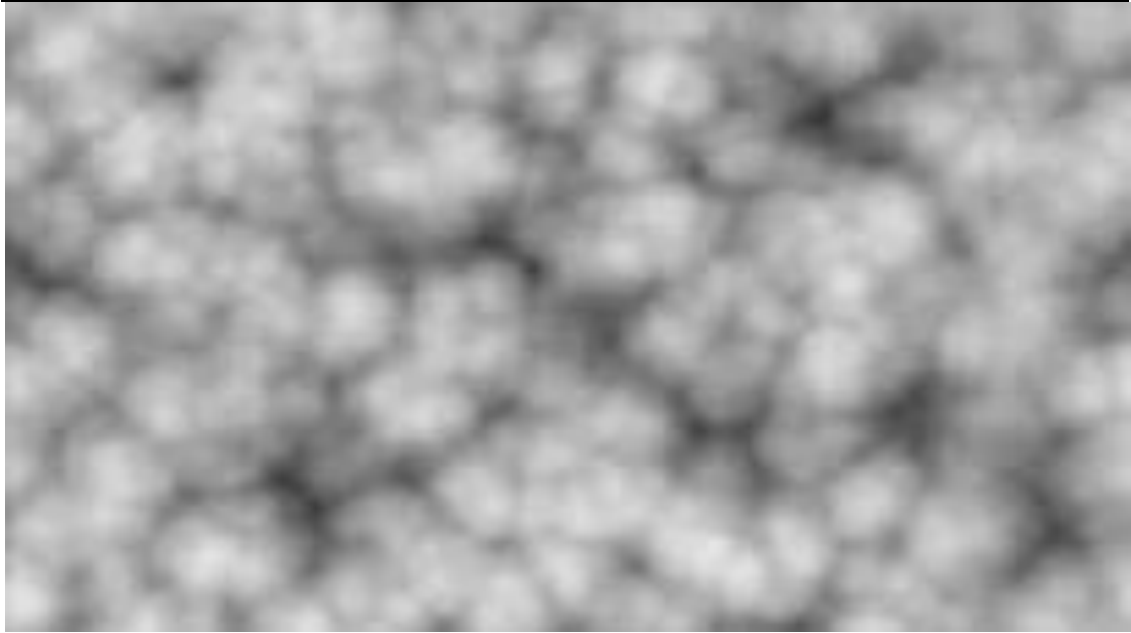


图 3-15 Schneider SIGGRAPH 2015 噪声结果

3.4 本章小结

本章主要介绍体积云建模。首先介绍根据体纹理生成体积云的方法和过程，然后介绍了根据 3D 噪声生成体积云的方法和过程，其中详细阐述了如何使用 Perlin 噪声和 Worley 噪声，以及结合密度-高度函数和时间函数生成体积云体积场的方法。

第4章 体积云渲染

本章提出一种基于光线步进的体积云渲染的方法，在完成体积云渲染的基础上，根据云的真实表现对光照近似进行优化，提升视觉效果，并对渲染算法进行优化，提高渲染效率。在有了体积云模型后，接下来的问题就是如何根据云的体积场以及渲染部分需要根据建模，计算光照和阴影，并最终将云在屏幕上显示出来。为了模拟逼真的云的效果，在渲染时需要一个好的方法根据建模对光照进行计算，而完全基于物理的光照需要大量的采样，无法实现实时渲染，为了实现逼真的光照效果，我们通过一系列光照定律来对体积云的光照进行近似计算。当光线在穿过云层时会发生内散射、外散射和吸收，我们首先通过比尔-朗伯定律对其进行近似，比尔-朗伯定律是分光光度法的基本定律，比尔-朗伯定律指出，我们可以根据传播介质的光学厚度确定某一点的光辐射度。而在云中的内散射具有各向异性，其中光向前散射的可能性更高，我们使用 HG 相位函数来对各向异性散射进行近似，HG 相位函数最初用于在银河尺度上进行光的计算，如今也用于对光照中的各向异性散射进行可靠的近似。除此之外云还具有暗边效应，即当我们的视线接近太阳光的光线方向时，可以很明显的看到云的边缘变暗，而比尔-朗伯定律是一个消光函数模型，只通过比尔-朗伯定律无法近似当云朝向太阳方向时，发生的这种散射照明现象，我们针对这种现象进行了额外的近似。最终根据比尔-朗伯定律、HG 相位函数和云的暗边效应对体积云的光照进行近似计算。有了光照的近似计算结果后我们使用 Ray Marching 方法实现体积云的渲染，Ray Marching 的原理是对体积场进行连续采样，通过每个采样点的密度和光照与该条射线当前累积的颜色值进行混合，以离散值的叠加逼近连续体积场的外观，同时每条光线可以并行计算，实现对体积场逼真的实时渲染。在此基础上我们根据体积云的特征，对 Ray Marching 算法进行优化，提高渲染效率。最终通过比尔-朗伯定律、HG 相位函数和云的暗边效应对体积云光照进行近似计算，使用改进的光线步进算法实现体积云的高逼真实时渲染。

4.1 体积云光照计算

考虑单个粒子（分子，灰尘，水滴等）光子将与其表面发生作用并反弹。例如云，想象一个微小的水滴：该水滴会折射和反射光线，但是几乎不会吸收他们，这也是为什么云通常非常明亮的原因：他的反照率（入射光反射比）几乎是100%。

如果光线传播进入一个高密度粒子填充的体积场，光子与每个粒子发生折射和反射，最终产生外散射、吸收和内散射现象。粒子截面 $a = \pi r_e^2 (m^2)$ ，其中 r_e 是有效半径（通常云中的水滴半径为 $2-20\mu m$ ）。而吸收/散射截面： $\sigma = aN_0 (m^{-1})$ ，其中 N_0 是水滴的密度（对于云而言，取决于云的类型和空气污染程度，通常为 10^8-10^9 每立方米）。

用比尔-朗伯定律表示光被吸收的量正比于光程中产生光吸收的分子数目：

$$\tau(d) = e^{-\sigma_t d} \quad (4-1)$$

这表示光子延长度为 d 的路径穿过介质不碰到粒子的概率，也代表介质的透明度。宏观上来讲光被透明介质吸收的比例与入射光的强度无关；在光程上每等厚层介质吸收相同比例值的光。

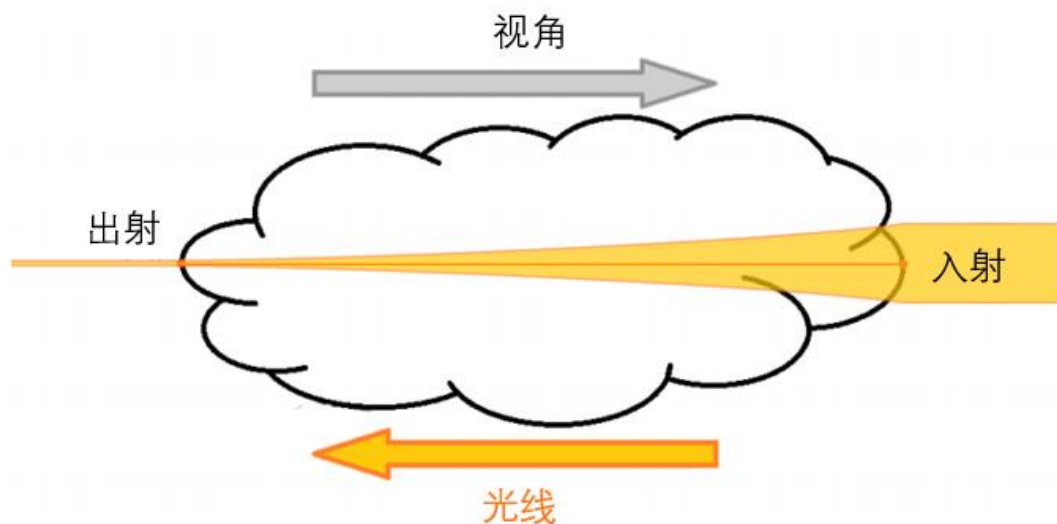


图 4-1 云中光的吸收示意图

因此，如图 4-1 所示，任何辐射进入云层 Δx 的距离，其强度便会降低：

$$L(x, \omega) = e^{-\sigma_t \Delta x} L(x + \Delta x \omega, \omega) \quad (4-2)$$

其中， ω 代表视角方向， x 是光线离开体积的位置， Δx 是进入云层和离开云层之间的距离。

当通过分子和粒子撞击时会失去能量，但是也会有能量从其他地方经过撞击向视角方向过来，而在云中的内散射具有各向异性，其中光向前散射的可能性更高。这会增加一些能量甚至可能弥补消光带来的能量损失，我们通过相位函数（The Phase Function）来估计光子从一个特定方向反弹到另一个方向的概率，特别是我们的视角方向。这里我们使用 Henyey-Greenstein 相位函数来代替使用复杂的相位函数，该相位函数由多个简化函数组合而成：

$$p_{HG}(\theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g \cos(\theta))^{3/2}} \quad (4-3)$$

其中 g 和之前的 g 一样，为倾向散射方向参数，取值范围为 $[-1, +1]$ 。

相位函数可以使用权重累计计算，假设权重之和为 1：

$$p(\theta) = w_0 \cdot p_{HG}(\theta) + w_1 \cdot p_{HG}(\theta) + (...) + w_i \cdot p_{HG}(\theta) \quad \sum w_i = 1 \quad (4-4)$$

比尔-朗伯定律是一个消光函数模型，它描述的是光的辐射度如何在深度上衰减。只通过比尔-朗伯定律无法近似当云朝向太阳方向时，发生的散射照明效果。当我们的视线接近太阳光的光线方向时，可以很明显的看到云的边缘变暗，这个效果称为云的暗边效应。暗边效应在球形、密集的云层区域中十分明显，导致在每个凸起之间的凹痕看起来比凸起的部分更加明亮，而实际上凸起本身距离太阳更近，这与比尔-朗伯定律正相反，如图 4-2 所示，当我们和太阳光同一方向观察云景时也会发生这种现象。

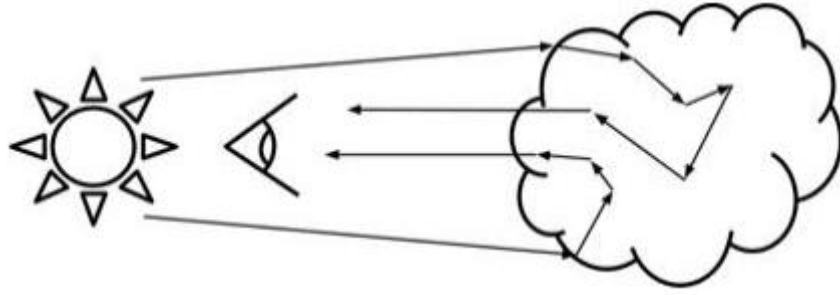


图 4-2 使光线方向逆转的散射

在 HG 相位函数中, 通常取 $g=0.2$, 也就是说更多的光会沿着原始光线方向进行向前散射。所以需要存在足够大的光学深度才能有机会使光子经过多次散射转向。然而云的边缘周围的路径的光学深度一般不足以使光子转动光子, 它们一般存在于云的内部。因此, 在它进入我们的眼睛之前, 通过比尔-朗伯定律可得大部分光子会被吸收。而云中的凹痕处是一个例外, 它提供了一个进入云内部的捷径, 其中存在具有相对大的光学深度的光子路径, 允许光子以低密度的方式快速逃逸, 从而使得凹痕比周围的凸起处更亮。

这种现象是一种概率现象, 当我们观察云中这些凸起区域中的一个, 其角度与太阳光的一组光线相同, 我们在一个凸起的表面下方和一个凹痕处下方相同深度分别采样一个点进行比较, 凹痕下方的点有更大概率将通过的光子经过散射穿出云表面, 即具有更高的散射潜力。所以从概率上来讲, 凹痕处应该比凸起处更亮一些。因此我们需要一个新的函数来对这种暗边效应做一个较为准确的近似, 同时尽量不增加额外的采样开销。因为这个现象在结果上实际与比尔-朗伯定律是相反的, 即在光学深度不够大时, 大部分光子无法经过多次散射进入我们的眼睛, 经尝试将该效应用比尔-朗伯定律取反并乘以一个经验参数 δ 来表示, 可以得到很好的近似效果, 即 $\tau(d) = 1 - e^{-\delta\sigma_t d}$ 。

然后将暗边效应与比尔-朗伯定律相结合并将其结果归一化得:

$$\tau(d) = \text{Normalize} \left((e^{-\sigma_t d})(1 - e^{-\delta\sigma_t d}) \right) \quad (4-5)$$

在计算在介质内部追踪距离 D 时, 我们定义 $L(x, \omega)$ 是沿方向 ω 的距离 x 处的

辐射度:

$$L(x, \omega) = \int_0^D e^{-\tau(x, x')} \sigma_s(x') \left[\int_{\Omega_{4\pi}} p(\omega, \omega') L_i(x', \omega') d\omega' \right] dx' \quad (4-6)$$

$$\tau(x, x') = \int_x^{x'} \sigma_t(t) dt$$

其中 $e^{-\tau(x, x')}$ 部分代表消光, 它使用光学深度 (Optical Depth) 来表示, 代表从 x 到 x' 的路径上累积的消光系数乘以该路径的长度。我们将追踪路径分割成若干小路径 Δx , 如果假设 σ_t 在长度为 Δx 的路径上为常数, 则 $\tau(x)$ 可以化简为 $\tau(x) = \sigma_t(x)\Delta x$ 。

而 $\left[\int_{\Omega_{4\pi}} p(\omega, \omega') L_i(x', \omega') d\omega' \right]$ 部分代表内散射。我们获取相位函数和来自所有可能方向的光 $L_i(x', \omega_i)$ 的复数积分。麻烦的部分是 $L_i(x', \omega_i)$ 需要上面式子本身来计算。这是一个递归关系, 每层递归计算一个新的散射时间。这也是为什么多次散射是非常耗费系统资源的原因并且大多数时候我们需要通过使用一个简化的环境项来对其近似实现实时渲染。

为了简化计算复杂度, 我们将光源分为三类: 来自太阳的平行光、来自太阳的环境光、来自天空的环境光。

于是光辐射度计算方程变为:

$$L(x, \omega) = \int_0^D e^{-\tau(x, x')} \sigma_s(x') [p_{sun}(\omega, \omega_{sun}) L_{sun}(x', \omega_{sun}) + p_{amb} L_{amb}] dx' \quad (4-7)$$

该积分可以理解为以下四个步骤:

- 1、计算来自太阳和周围环境在到达 x' 处的入射光。
- 2、通过 $\sigma_s(x')$ 发生散射。
- 3、让观察者只看到 x 到 x' 之间的一小部分光线。
- 4、开始重新计算下一小步 dx' 。

于是有新的近似方程:

$$L(x, \omega) \approx \sum_{i=1}^N e^{-\tau(x, x+i\Delta x)} \sigma_s(x+i\Delta x) [p_{sun}(\omega, \omega_{sun}) L_{sum}(x+i\Delta x, \omega_{sun}) + p_{amb} L_{amb}] \Delta x \quad (4-8)$$

其中 $\Delta_x = D/N$, N 为在介质内部的离散步数。

对于 $e^{-\tau(x, x+i\Delta_x)}$ 部分, 有 $e^{-\tau(0, \Delta)} = e^{-\tau(0, \frac{\Delta}{2})} \cdot e^{-\tau(\frac{\Delta}{2}, \Delta)}$ 。

以固定步长 Δ_x 进行递归, 可以得到:

$$e^{-\tau(x, x+i\Delta_x)} \approx \prod_{j=1}^{i-1} e^{-\sigma_t(x+j\Delta_x)\Delta_x} \quad (4-9)$$

我们看到它变成一个以每个小切片的不透明度乘积的形式进行不透明度的累积。我们可以将其保存在寄存器中并且在每个新一步时相乘。最终着色器的方程变为:

$$L(x, \omega) \approx \sum_{i=1}^N E x_i \sigma_s(x + i\Delta_x) [p_{sun}(\omega, \omega_{sun}) L_{sun}(x + i\Delta_x, \omega_{sun}) + p_{amb} L_{amb}] \Delta_x \quad (4-10)$$

其中 $E x_i = E x_{i-1} e^{-\sigma_t(x+i\Delta_x)\Delta_x}$, $E x_0 = 1$ 。

接下来我们通过 Ray Marching 算法结合上述光照近似方程和体积场密度采样对体积云进行渲染。

4.2 光线步进

4.2.1 光线步进原理

光线步进^[39] (Ray Marching) 是从屏幕发射射线, 每次通过光线前进一定步长采样计算, 来对体积场进行近似的一种体绘制^{[40][41]}算法。

Ray Marching 可以看作 Ray Tracing 的一种实现, 主要针对“光线与物体求交”这一步, 每次让光线前进一定步长, 并检测当前光线是否位于物体表面, 据此调整光线前进幅度, 直到抵达物体表面, 再按照一般光线追踪的方法计算颜色值。Ray Marching 和 Ray Casting 类似, 都是从屏幕发射射线, 然后求射线和物体的焦点, 但是光线投射是一次性算出交点, 而光线步进是一步步的前进不断的向交点趋近, 光线步进中的物体使用一种距离场函数来表示。

距离场是三维空间中一个标量场，任意一点的数值代表离该点最近的模型表面的点的距离下界。

有向距离场（Signed Distance Field，简称 SDF）是某度量空间 X 中的一个集合 Ω 的函数，决定 X 中任一点到 Ω 边界 $\partial\Omega$ 的距离，并且由 x 是在 Ω 内还是 Ω 外确定其 SDF 的正负号：当 x 在 Ω 内时，SDF 为正；当 x 在 Ω 外时，SDF 为负。

假设 d 是空间 X 的一种度量，那么 SDF 用数学公式表达：

$$\begin{aligned} f(x) &= d(x, \partial\Omega) & \text{if } x \in \Omega \\ &= -d(x, \partial\Omega) & \text{if } x \in \Omega^c \end{aligned} \quad (4-11)$$

其最初用于字体绘制，通过距离场对字体进行无损放大。后来也用于 Ray Marching 以及环境光遮蔽。

如图 4-3 所示，通过距离场可以很方便地估计 Ray Marching 中每次光线应当前进的幅度，只需让光线前进当前点距离场大小的数值即可。简单几何图形的距离场是很容易计算的：例如球的距离场就是到球心距离减去球半径，立方体距离场就是到立方体六个面距离最小值。已知一个距离场，通过一些简单的数学手段即可以对其进行编辑，例如求交、求并、扭曲等等，而相同的编辑操作在三角面片上完成起来是相当复杂的。对于一些复杂的模型，例如分形，它们的距离场也是可求的；而即使是类似 Perlin Noise 这样没有数学表达式的噪声模型，也可以通过某些手段对距离场做近似。

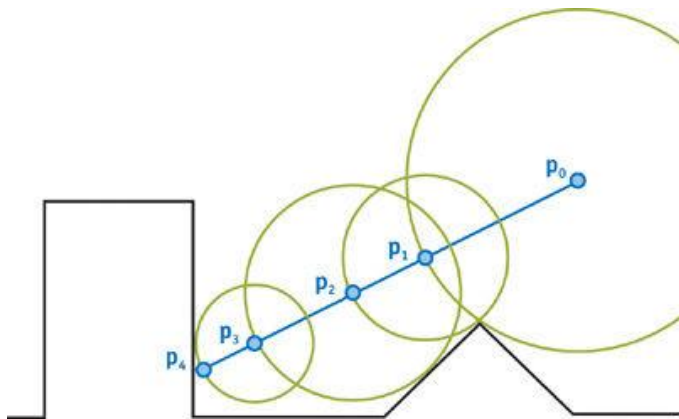


图 4-3 距离场示意图

（图片来自书《GPU Gem 2》Figure 8-5）

通过距离场函数可以得到当前点的位置和物体的最近距离，如果距离趋向于0，就说明到达了交点，每次前进的步长等于计算的最近距离，这样可以更快的趋近与交点。

如图 4-4 所示，在对体积云进行渲染时，Ray Marching 的机理使得我们可以对云的体积场做采样，以离散值的叠加逼近一团连续体积场的外观，从而渲染出云的效果。首先对于一个 3D 的体积场，从相机发射 N 条射线，当射线处在体积场中时，每个步长进行一次采样，根据第 3 章基于 3D 噪声建模的体积云体积场密度函数的到该点的密度采样值，接下来根据 4.1 中基于比尔-朗伯定律、HG 相位函数、暗边效应的体积云光照辐射度方程计算光照，然后和该条射线当前累积的颜色值进行混合。因为光路是可逆的，从光源射出的光线经过散射，最终进入摄像机的效果等同于从摄像机发出的射线进行着色和采样，这与 Ray Tracing 的道理是一样的。并且因为每条射线的计算都是独立的，该方法可以在 GPU 上并行实现。

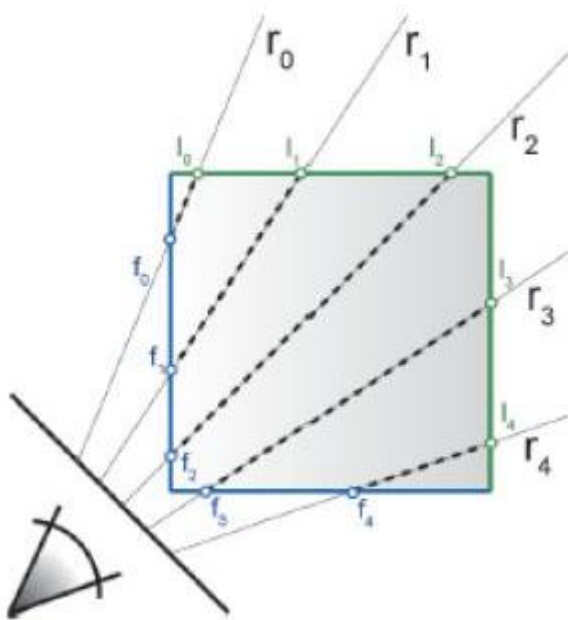


图 4-4 Ray Marching 示意图

(图片来自互联网 体绘制 (Volume Rendering) 概述-CSDN 博客)

体积云的 Ray Marching 算法的伪代码如下：

```
Function RayMarching ( StartPosition , directionFromEye , steps , stepSize )  
    entry = RayIntersect ( StartPosition , directionFromEye );  
    t = 0.0f;  
    for ( i = 0 ; i < steps ; ++i )  
        position = entry + directionFromEye * t;  
        density = GetDensity ( position );  
        accumulation += EvaluateLight ( position , density );  
        t += stepSize;  
    return accumulation;
```

其中 GetDensity 函数为第 3 章基于 3D 噪声建模的体积云体积场密度函数，EvaluateLight 函数为 4.1 节中基于比尔-朗伯定律、HG 相位函数、暗边效应的体积云光照辐射度计算方程。

4.2.2 光线步进优化

其中在 Ray Marching 过程中，存在着许多可以对其进行加速的剪枝和优化，其中包括可变步长和提前跳出等。

可变步长：如果在 Ray Marching 过程中采用固定步长，且步数不够多，就会出现条纹装的瑕疵。同时因为人们的注意力会更关注于近距离的眼前的事物，对于稍远处的事物不用和近距离的事物一样细化，所以远处的采样点对近处的影响较小，则可以令每次采样的步长随着当前采样距离的增加而增加。可以设定一个经验参数 δ ，假设 t 为当前采样距离， Δd 为最小步长，则有： $\Delta t = \max(\Delta d, \delta t)$ 。这样不仅能够消除采用固定步长时产生的条纹装瑕疵，还可以在极小降低效果的情况下有效的减少采样步数，从而增加渲染效率。

而当采样时，某条光线上的采样密度连续为接近 0 的低密度的话，我们可以不去对这些采样点继续进行对于太阳光方向的采样，同时增加步长，直到采样到

密度超过给定阈值时再使用常规步长继续进行采样。

提前跳出：在 Ray Marching 时并不是所有光线都要进行规定的采样步数，光线在很多情况下可以提前跳出，结束采样过程。一种情况为当采样累积的不透明度为 1 时，此时在该路线的剩余路径上已经完全不透光，意味着没有剩余的光子可以继续在这条路径上继续运动，此时可以提前跳出采样，直接进行下一步计算。

同样当采样位置已经超出体积场范围时，也可以提前跳出采样，直接进行下一步计算。当某条采样的光线上直接可以看到天空，即这条线上云的密度为 0 时，这种情况下同样可以提前跳出采样。

改进后的 Ray Marching 算法伪代码如下：

```
Function RayMarching( StartPosition , directionFromEye , steps , stepSize )
    entry = RayIntersect ( StartPosition , directionFromEye );
    exit = RayIntersect ( EndPosition , directionFromEye );
    for ( i = 0 ; i < steps ; ++i )
        if ( t > Distance ( entry, exit ) ) break;
        if ( accumulation.opacity == 1 ) break;
        position = entry + directionFromEye * t;
        density = GetDensity ( position );
        if ( density == 0 )
            cnt++;
            t += CalculateSetpSize ( t , stepSize , cnt );
        else
            cnt=0;
            accumulation += EvaluateLight ( position , density );
            t += CalculateSetpSize ( t , stepSize , cnt );
    return accumulation;
```

其中 GetDensity 函数为第 3 章基于 3D 噪声建模的体积云体积场密度函数，EvaluateLight 函数为 4.1 节中基于比尔-朗伯定律、HG 相位函数、暗边效应的体

积云光照辐射度计算方程, `CalculateSetpSize` 函数为基于当前采样距离和基础步长计算可变步长。

最终通过比尔-朗伯定律、HG 相位函数和云的暗边效应对体积云光照进行近似计算, 使用改进的光线步进算法实现体积云的高逼真实时渲染。

4.3 实验结果

本章在实现体积云渲染的基础上, 对云的“暗边效应”这种现象使用了额外的光照近似计算, 在视线接近太阳光的光线方向时有更真实的视觉效果, 同时对于体积云渲染的方法光线步进根据体积云本身的特点进行可变步长、提前跳出等一系列优化, 大幅提高体积云的渲染效率。

本论文方法在 PC 端部署, 所使用的显卡型号为 GTX1060, 使用系统为 windows10 系统, 使用 Unreal 平台版本为 4.22.3, 使用 HLSL 和 C++ 进行编写。

首先简单展示基于模型以及基于体纹理生成的云的视觉效果, 如图 4-5、4-6 所示。

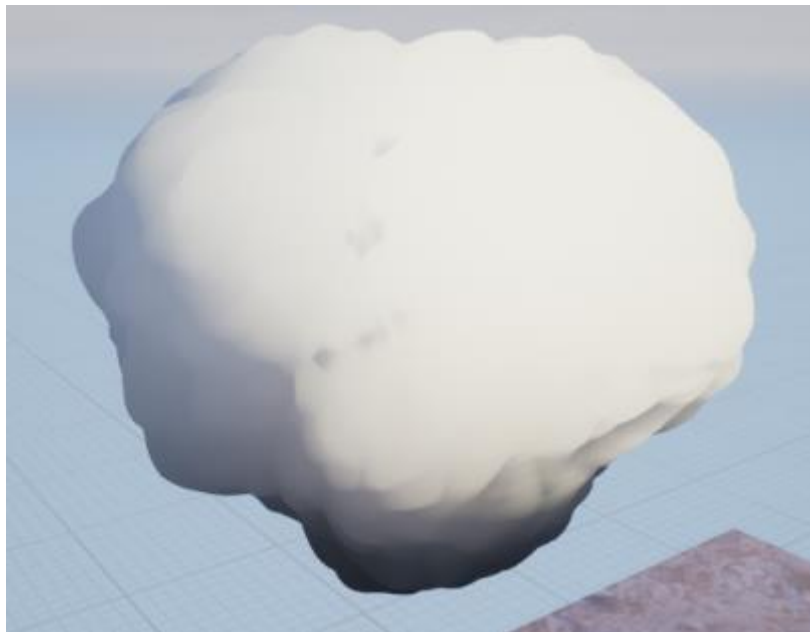


图 4-5 基于模型生成的云视觉效果



图 4-6 基于体纹理生成的体积云视觉效果

接下来，我们将展示本文提出的体积云实时渲染方法结果。

在渲染效率方面，我们将本文提出的体积云实时渲染方法与 Schneider^[16]的体积云渲染方法作对比，该方法发表于 SIGGRAPH2015，并在《地平线：零之黎明》游戏中使用，获德国科隆游戏展最佳视觉奖，其体积云渲染方法常用与制作 3A 游戏。在不考虑分帧渲染的优化的情况下，其渲染整个屏幕一帧需要的时间平均为 20ms，能够达到 50 帧每秒的效率渲染。相比之下我们在渲染时根据体积云的特点对光线步进算法做了更多的优化策略，包括可变步长和提前跳出等，最终渲染整个屏幕一帧的时间平均为 2ms，在 PC 端实现了更高效率的体积云渲染。

在视觉效果方面，我们将本文提出的体积云实时渲染方法同样和 Schneider^[16]的体积云渲染方法作对比，并同时与现实中自然景观的云相比较，展示其视觉效果，如图 4-7~12 所示。其中前者的方法使用了更多的额外参数以及后处理来对云

的视觉效果进行优化，而这里我们更关注体积云渲染方法本身。这里我们主要关注云的边缘的通透效果，以及太阳光穿过云时的表现。在这些方面本文方法与前者的方法相差无几，视线接近太阳光的光线方向时，由于我们对于“暗边效应”进行了额外的近似，这种情况下在光照方面有更好的表现。



图 4-7 本文实现体积云的视觉效果 1



图 4-8 《地平线：零之黎明》中云的视觉效果 1



图 4-9 现实自然景观中云的视觉效果 1

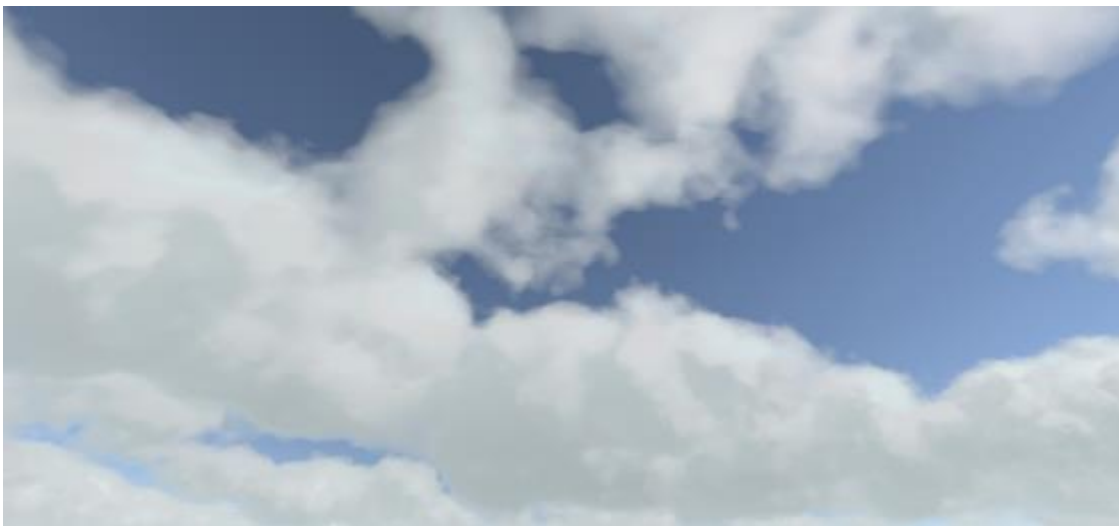


图 4-10 本文实现体积云的视觉效果 2



图 4-11 《地平线：零之黎明》中云的视觉效果 2



图 4-12 现实自然景观中云的视觉效果 2

4.4 本章小结

本章主要介绍体积云渲染。首先通过比尔-朗伯定律、相位函数、暗边效应这些云渲染用到的光照函数和模型对光辐射度进行了推导和计算，接下来详细阐述了结合上述光照模型使用光线步进算法生成体积云的方法，对光线步进算法进行优化分析，最后对结果进行展示。

第5章 体积云在移动端加速方法

本章提出一种体积云在移动端的加速方法，在目前还没有人实现移动端体积云实时渲染解决方案的情况下，尝试使用基于多边形模型的方法对体积云进行建模，并且通过一系列渲染算法的优化，实现体积云在移动端的实时渲染。随着移动端的普及和移动端性能的快速发展，人们对于自然景观在移动端的渲染效果的要求也随之提高，我们尝试在适当降低精度的情况下实现动态体积云的实时渲染模拟。但是无论是通过体纹理还是 3D 噪声建模体积云进行 Ray Marching，对于移动端的性能和内存而言都是一个巨大的开销。为了解决这个难题，提高渲染效率，大幅减少光线步进时间，我们基于多边形模型对体积云体积场的建模。基于 Hypertexture 的思想，我们尝试利用在像素着色器中能获取到的参数和云的特征对云的体积场密度进行近似估计。而云的形状是不规则的，除了基于多边形模型对其进行建模，还需要噪声的加入，对体积场的密度估计函数进行扰动。由于移动端性能的限制，我们在使用生成速度最快的 Value 噪声。有了体积云的体积场密度估计函数后，我们基于多边形模型对体积云进行渲染，我们仍然使用光线步进算法，由于光线步进算法通常是基于体积场来实现，所以我们在此基础上对其进行改进，使其能够基于模型来实现体积云的渲染。有了光照之后，一个逼真的云的模拟同样少不了阴影，PC 端实现方法是对于每个采样点再沿光照方向进行多次采样，这个方法会使时间复杂度成倍增长，对于移动端的性能而言难以实现，这里我们将阴影计算简化，同样基于模型使用 Shadow Map 来计算阴影。Shadow Map 是一种常用的实时阴影的快速生成方法，通常用来生成平行光的阴影。最终基于多边形模型对云的体积场生成密度估计函数，结合由 Value 噪声对体积云的体积场进行建模，使用改进的基于模型表面的光线步进算法进行渲染，以及使用 Shadow Map 加速阴影渲染，实现体积云在移动端的加速。

随着移动端的普及和性能的快速发展，人们对于移动端的渲染效果的要求也随之提高。我们尝试在移动端不具备 PC 端的高性能的情况下，能够在适当降低

精度的情况下提升速度,使其和 PC 端一样能够实现动态体积云的实时渲染模拟。

人们通常在观察云的时候,对于云的效果是否逼真,更多会关注于其表面一层的效果,因为在云的表面附近,云的密度和相应的光照变化会比较明显,而对于中间部分,关注度较低,对于效果影响相对较小,在这里我们使用密度函数来对云的体积场进行描述,将云看作外部是密度 0-1 变化的软体区域,中心部分是密度为 1 的实体,这个想法的来源基于超纹理 (Hypertexture)。

5.1 超纹理

在计算机图形学中,传统上将对象建模为具有无限薄的边界面的集合。然后通常将计算出的或数字化的纹理或位移映射到表面上以增强真实感。但是,有着仅将对象边界视为表面存在的局限性。

许多物体(例如毛皮或机织材料)具有复杂的定义,该定义几乎不可能用表面模型来描述。对于其他物体,例如侵蚀的材料或流体,有着人工定义的非常复杂的表面边界,其通常更容易用体积进行描述。还有一些其他物体,例如火焰,云层或烟雾,实际上根本没有定义明确的边界表面。

许多此类对象的外观可以通过一些函数直接描述,并通过对三维空间某些区域的采样进行评估。Perlin 基于程序纹理的生成研究,并和他的一名学生在 SIGGRAPH 1989 上发表了一篇文章^[42],提出了超纹理(hypertexture),通过体绘制中的思想与一些新扩展的模型相结合,使用噪声、fBM 和 Ray Marching 实现了各超纹理。

Hypertexture 模型本质上是在整个体积区域而不是仅在表面对物体进行模拟。该方法能够获得诸如毛发、毛皮、火、玻璃、流体流动和侵蚀效应等形状和纹理的视觉逼真的表示。

代替将对象建模为连接的曲面,Hypertexture 将对象建模为密度的分布。描述了一种生成简单的基本形状的密度分布的机制。这些基本形状的硬区域是完全实心的,软体区域是不确定的。每当处于软体区域时,可以使用 fBM 噪声,从而灵

活地创建和操纵软体区域部分的体积形式。即该区域是可延展的，我们可以以可控的方式收缩、拉伸、扭曲或以其他方式变形模拟物。

描述 Hypertexture 需要使用以下两个概念：

物体密度函数（Object Density Function） $D(x)$ ，范围为 $[0,1]$ ，用来描述一个 3D 物体在 R^3 空间上的任意一点的密度。一个物体的软体区域为所有 x 的满足 $0 < D(x) < 1$ 的集合。

密度调制函数（Density Modulation Function，简称 DMF） f_i ，用来控制物体软体区域部分的密度。每个 DMF 用来控制物体的空间特征，DMF 集合就是一个体积建模工具。

Hypertexture 通过对物体的密度函数 $D(x)$ 连续应用一系列 DMF f_i 实现：

$$H(D(x), x) = f_n \left(\dots, f_2 \left(f_1 \left(f_0(D(x)) \right) \right) \right) \quad (5-1)$$

DMF f_i 可以为三种类型：位置相关，位置无关和几何相关。与位置有关的 DMF 为 $f(x)$ ，与位置无关的 DMF 为 $f(k)$ ，其中 k 为标量，而与几何形状有关的 DMF 可能取决于 x 以外的变量，例如 x 附近的密度梯度。

软体是 R^3 上的密度函数 $D(x)$ ，其中 D 在物体内部值为 1.0，在物体外部值为 0.0，在两者之间的软体区域中值为 $0.0 < D < 1.0$ 。例如，以半径 r 和柔软度 s 为中心的球体的密度函数定义：

$$\begin{aligned} D_{(c,r,s)}(x): \\ r_0^2 &:= (r + s/2)^2 \\ r_1^2 &:= (r - s/2)^2 \\ r_x^2 &:= (x_x - x_c)^2 + (y_x - y_c)^2 + (z_x - z_c)^2 \\ D &:= \text{if } r_x^2 \leq r_1^2 \text{ then } 1.0 \text{ else} \\ &\quad \text{if } r_x^2 \geq r_0^2 \text{ then } 0.0 \text{ else } (r_0^2 - r_x^2)/(r_0^2 - r_1^2) \end{aligned} \quad (5-2)^{[42]}$$

其中 r_0 是外部（ $D = 0$ ）边界， r_1 是内部（ $D = 1$ ）边界， r_x 是点 x 处球体的半径。

其渲染是在 GPU 上通过 Ray Marching 并行实现，从视点发射无数条射线，每条射线穿过物体对不透明度进行累计计算，即对密度进行积分来实现的，Ray

Marching 算法在 4.2 中已经详细阐述。

但是无论是通过体纹理还是 3D 噪声建模体积云进行 Ray Marching, 对于移动端的性能和内存而言都是一个巨大的开销。为了解决这个难题, 提高渲染效率, 大幅减少光线步进时间, 本文尝试基于多边形模型实现体积云体积场的建模。其中模型的生成可以通过 Houdini 实现, 具体的方法与 3.1 节中的体纹理生成类似, 但是最后导出的仅为多边形模型, 其中不包含体素信息。

5.2 基于模型的密度估计函数

基于 Hypertexture 的概念, 将云看作软体, 定义 R^3 上的密度函数 $D(x)$, 其中 D 在物体内部值为 1.0, 在物体外部值为 0.0, 在两者之间的软体区域中值为 $0.0 < D < 1.0$ 。云基于模型产生的密度函数体积场期望规则: 密度期望函数为模型表面密度为 0, 从模型表面到模型内部软体区域部分密度从 0 到 1 递增, 中间硬区域部分密度为 1。基于式(5-2)对于模型 M , 柔软度 s 的云的密度期望函数定义为:

$$\begin{aligned}
 & D_{(M,s)}(x): \\
 & r_0^2 := |PC|^2 = (x_p - x_c)^2 + (y_p - y_c)^2 + (z_p - z_c)^2 \\
 & r_1^2 := ((1-s)r_0)^2 = (1-s)^2 |PC|^2 \\
 & \quad = (1-s)^2 \left((x_p - x_c)^2 + (y_p - y_c)^2 + (z_p - z_c)^2 \right) \quad (5-3) \\
 & r_x^2 := |XC|^2 = (x_x - x_c)^2 + (y_x - y_c)^2 + (z_x - z_c)^2 \\
 & D := \text{if } r_x^2 \leq r_1^2 \text{ then } 1.0 \text{ else} \\
 & \quad \text{if } r_x^2 \geq r_0^2 \text{ then } 0.0 \text{ else } (r_0^2 - r_x^2) / (r_0^2 - r_1^2)
 \end{aligned}$$

其中, ∂M 为模型 M 的边界, 点 X 为模型 M 内任意一点, 点 C 为模型 M 中心点, 点 P 为射线 CX 与模型边界 ∂M 的交点。

可以看到其中涉及射线与平面求交的操作, 这将会产生很大的开销, 在移动端无法实现, 所以我们尝试利用在像素着色器中能获取到的值对云的体积场密度进行近似估计, 使其与上面定义的云的密度期望函数尽可能相似。

首先基于 Bounding box, 将模型大小归一化到单位立方体, 使其密度分布更加均匀, 接下来进行的近似能够更加准确。

在像素着色器中,我们能够获取该像素 P_0 的世界坐标、相机方向 ω 和模型中心点 C 坐标信息。对于当前采样点 X ,有 $X=P_0+\omega D$,其中 D 为在介质内部的追踪距离。

我们假设云的形状较为规则,密度变化速率均匀,则有 $|PC|\approx|P_0C|$,基于式(5-3),可得密度估计函数 $\widehat{D}_1$:

$$\begin{aligned}\widehat{D}_{1(M,s)}(x): \\ r_0^2 &:= |PC|^2 \approx |P_0C|^2 = (x_{p_0} - x_c)^2 + (y_{p_0} - y_c)^2 + (z_{p_0} - z_c)^2 \\ r_1^2 &:= ((1-s)r_0)^2 \\ r_x^2 &:= |XC|^2 = (x_x - x_c)^2 + (y_x - y_c)^2 + (z_x - z_c)^2 \\ D &:= \text{if } r_x^2 \leq r_1^2 \text{ then } 1.0 \text{ else} \\ &\quad \text{if } r_x^2 \geq r_1^2 \text{ then } 0.0 \text{ else } (r_0^2 - r_x^2)/(r_0^2 - r_1^2)\end{aligned}\tag{5-4}$$

其中, ∂M 为模型 M 的边界,点 X 为模型 M 内当前采样点,点 C 为模型 M 中心点,点 P 为射线 CX 与模型边界 ∂M 的交点,点 P_0 为相机射线与模型边界 ∂M 交点。

我们假设云的形状不规则,但密度变化速率对于模型边界上的点 P 在向模型中心方向 $\overrightarrow{PC}$ 最快,沿其他方向变化速率均匀递减,即密度的二次导数为常数,则有 $\frac{|PX|}{|PC|} = \frac{|P_0X|\dot{\text{dot}}(\overrightarrow{P_0X},\overrightarrow{P_0C})}{|P_0C|}$,又有 $|PC|=|PX|+|XC|$,基于式(5-3),可得密度估计函数 $\widehat{D}_2$:

$$\begin{aligned}\widehat{D}_{2(M,s)}(x): \\ r_0^2 &:= |PC|^2 \approx \frac{|XC| \cdot |P_0C|}{|P_0C| - |P_0X|\dot{\text{dot}}(\overrightarrow{P_0X},\overrightarrow{P_0C})} \\ r_1^2 &:= ((1-s)r_0)^2 \\ r_x^2 &:= |XC|^2 = (x_x - x_c)^2 + (y_x - y_c)^2 + (z_x - z_c)^2 \\ D &:= \text{if } r_x^2 \leq r_1^2 \text{ then } 1.0 \text{ else} \\ &\quad \text{if } r_x^2 \geq r_1^2 \text{ then } 0.0 \text{ else } (r_0^2 - r_x^2)/(r_0^2 - r_1^2)\end{aligned}\tag{5-5}$$

其中, ∂M 为模型 M 的边界,点 X 为模型 M 内当前采样点,点 C 为模型 M 中心点,点 P 为射线 CX 与模型边界 ∂M 的交点,点 P_0 为相机射线与模型边界 ∂M 交点。

我们假设云的形状不规则,但密度变化速率对于模型边界上的点 P 在向模型

表面法向量方向 $\vec{n}$ 最快, 沿其他方向变化速率均匀递减, 即密度的二次导数为常数, 则有 $\frac{|PX|}{|PC|} = \frac{|P_0X| \cdot \text{dot}(\overrightarrow{P_0X}, \vec{n})}{|P_0C|}$, 又有 $|PC| = |PX| + |XC|$, 基于式(5-3), 可得密度估计函数 $\widehat{D}_3$:

$$\begin{aligned} & \widehat{D}_{3(M,s)}(x): \\ & r_0^2 := |PC|^2 \approx \frac{|XC| \cdot |P_0C|}{|P_0C| - |P_0X| \cdot \text{dot}(\overrightarrow{P_0X}, \vec{n})} \\ & r_1^2 := ((1-s)r_0)^2 \\ & r_x^2 := |XC|^2 = (x_x - x_c)^2 + (y_x - y_c)^2 + (z_x - z_c)^2 \\ & D := \text{if } r_x^2 \leq r_1^2 \text{ then } 1.0 \text{ else} \\ & \quad \text{if } r_x^2 \geq r_1^2 \text{ then } 0.0 \text{ else } (r_0^2 - r_x^2)/(r_0^2 - r_1^2) \end{aligned} \quad (5-6)$$

其中, ∂M 为模型 M 的边界, 点 X 为模型 M 内当前采样点, 点 C 为模型 M 中心点, 点 P 为射线 CX 与模型边界 ∂M 的交点, 点 P_0 为相机射线与模型边界 ∂M 交点。

对于以上几个密度估计函数 $\widehat{D}_i(x)$, 根据不同的权重 w_i 对其求和, 得到基于模型的体积云体积场的密度估计函数:

$$\widehat{D}(x) = \sum w_i \cdot \widehat{D}_i(x) \quad (5-7)$$

因为云的形状是不规则的, 除了基于生成的多边形模型对其进行建模, 还需要噪声的加入, 对体积场的密度估计函数进行扰动。由于移动端性能的限制, 我们在这里使用能够快速生成的 Value 噪声。

5.3 基于 Value 噪声的体积场扰动构建

Value 噪声与 3.2.1 节中的 Perlin 噪声生成原理类似, 它把原来的梯度替换成了一个简单的伪随机值, 不需要进行点乘操作, 而直接把晶格顶点处的随机值按权重相加即可。

和 Perlin 噪声一样, Value 噪声也是一种基于晶格的噪声, 我们通过以下步骤生成 Value 噪声:

1、定义一个晶格结构，每个晶格的顶点生成一个“伪随机”的值（Value）。对于二维的 Value 噪声来说，晶格结构就是一个平面网格，三维的就是一个立方体网格。

2、输入一个点（二维的话就是二维坐标，三维就是三维坐标，n 维的就是 n 个坐标），我们找到和它相邻的那些晶格顶点（二维下有 4 个，三维下有 8 个，n 维下有 $2n \times 2n$ 个），得到这些顶点的伪随机值。

3、使用缓和曲线计算权重和。

我们所使用的噪声希望可以尽量快速的生成，而 Value 噪声比 Perlin 噪声的实现更加简单，并且需要的乘法和加法操作也更少，它只需要得到晶格顶点的随机值再把它们按权重相加即可，所以在移动端我们选择使用 Value 噪声。

如图 5-1 所示，这里我们首先利用 Value 噪声产生了云的雏形，将其进行适当放大，此时的噪声仍然十分粗糙，缺少细节。



图 5-1 放大 4 倍后的 Value 噪声

湍流噪声是基于基本噪声经过一系列操作叠加而生成的噪声，由于其产生的效果与湍流相似，从而叫做湍流噪声，其定义为：

$$\sum_i \text{abs} \left(\frac{1}{2^i} \text{noise}(2^i x) \right) \quad (5-8)$$

利用上述湍流噪声的定义，使用 Value 噪声经过一系列变换和叠加，生成基于 Value 噪声的湍流噪声，生成更多细节，与云的形状相近，结果如图 5-2 所示。

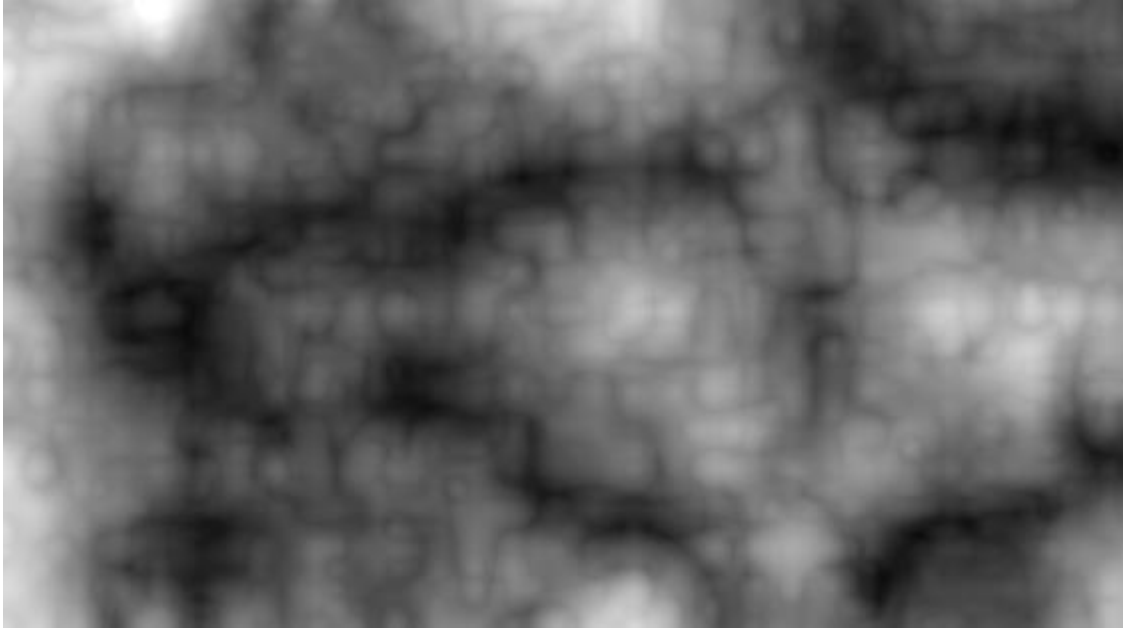


图 5-2 基于 Value 噪声生成的湍流噪声

结合密度估计函数，对其添加湍流噪声，最终我们得到基于模型的体积云体积场的密度函数：

$$D(x) = \hat{D}(x) \left(x(1 + \text{turbulence}(x)) \right) \quad (5-9)$$

有了体积云的体积场密度估计函数后，即完成了体积云基于模型的建模，接下来需要基于模型对体积云进行渲染，这里我们仍然使用光线步进算法，由于光线步进算法通常是基于体积场来实现，所以我们在此基础上对其进行改进，使其能够基于模型来实现体积云的渲染。

5.4 基于模型的光线步进算法

基于模型使用光线步进算法与 4.5 中提到的光线步进算法类似。因为 Ray Marching 的机理使得我们可以对体积场做采样，以离散值的叠加逼近一团连续体积场的外观。首先从相机发射 N 条射线，当射线和模型相交时，开始进行采样。当射线处在模型中时，每个步长采一次样，通过模型对体积云的体积场进行建模，建立密度估计函数方程，估计出 Ray Marching 过程中当前点的密度采样值，并计算光照，然后和该条射线当前累积的颜色值进行混合。因为光路是可逆的，从光源射出的光线经过散射，最终进入摄像机的效果等同于从摄像机发出的射线进行着色和采样，同样基于 GPU 每条射线的独立计算来实现。

同样当采样累积的不透明度为 1 时，说明此时在该路线的剩余路径上已经完全不透光，意味着没有剩余的光子可以继续在这条路径上继续运动，此时可以提前跳出采样，这样可以避免对于被遮盖部分进行不必要的计算，提高渲染效率。其中包括当射线到达云的硬区域部分时，即密度为 1 的区域，提前跳出采样。当我们软体区域部分为模型表面的薄薄一层，即柔软度 s 很小时，Ray Marching 采样步数大幅度减少，尽管渲染出的云的效果和逼真性有所下降，但是仍然可以较好的体现出云的动态效果、云表层的透光效果。

基于模型的 Ray Marching 伪代码如下：

```
Function RayMarching ( StartPosition , EndPosition , directionFromEye , steps , stepSize )
    for ( i = 0 ; i < steps ; ++i )
        if ( t > Distance( StartPosition , EndPosition ) ) break;
        if ( accumulation.opacity == 1 ) break;
        position = entry + directionFromEye * t;
        if ( isInHardRegin( position ) )break;
        else
            density = GetDensity ( position );
            if ( density == 0 )
                cnt++;
            t+= CalculateSetpSize ( t , stepSize , cnt );
```

```
else
    cnt=0;
    accumulation+= EvaluateLight ( position , density );
    t += CalculateSetpSize ( t , stepSize , cnt );
return accumulation;
```

其中 `GetDensity` 函数为 5.2 节与 5.3 节中基于模型建模的体积云体积场密度函数, `EvaluateLight` 函数为 4.1 节中基于比尔-朗伯定律、HG 相位函数、暗边效应的体积云光照辐射度计算方程, `CalculateSetpSize` 函数为基于当前采样距离和基础步长计算可变步长。。

有了光照之后,对于逼真的云的模拟同样少不了阴影,PC 端实现方法是对于每个采样点再沿光照方向进行多次采样,这个方法会使时间复杂度成倍增长,对于移动端的性能而言难以实现,我们将阴影计算简化,同样基于模型使用 `Shadow Map` 来计算阴影。

5.5 基于 Shadow Map 的阴影渲染

`Shadow Map`^{[43][44][45]}是一种常用的实时阴影的快速生成方法。通常用来生成平行光的阴影。对于体积云渲染,光源分为三类:来自太阳的平行光、来自太阳的环境光、来自天空的环境光。这里我们使用 `Shadow Map` 来替代 PC 端复杂的阴影计算,来加速阴影的渲染。

`Shadow Map` 算法的核心,是对任意给定的一个物体片段的点,判断其是否在阴影之中。在平行光的照射下,由于光线沿直线传播,一旦遇到遮挡,就会产生阴影。所以对于光线来说不可见的点,就是处在阴影中的点。这种可见性的判定,通常通过深度测试实现。即我们处在光源位置,按照光线传播方向的视角观察场景,计算场景中的物体的距离光源的距离,即在光源视角下的深度,并且记录每个位置上的最小值,即距离光源最近的点的位置,从而获得一张 `Shadow Depth`

Map。这个流程与普通相机获取深度图的流程基本一致，只是通过坐标系转换到光源位置来观察。在获取了 Shadow Depth Map 之后，用其进行判定：如果某个点在光源视角下的深度大于 Shadow Map 中对应位置的深度，就意味着它被某个物体遮挡，因此是在阴影中的；反之则不在阴影中。所以对于世界坐标系中的某点 p ，我们只要将其转移到光源空间下，比较它在光源空间下的深度，就可以判定它是否处于阴影中。

具体实现流程如下：

- 1、在光源位置放置相机。相机的投影方式有两种：正交投影（Orthographic Projection）和透视投影（Perspective Projection），通过相机投影方式的不同后续使用不同的转换矩阵。因为我们以光源的角度观察世界，光源是平行光，在其视角里世界没有透视关系，所以需要使用正交投影。然后确定视锥体（Frustum），我们需要恰到好处的视锥体大小来包含光源视角下的所有物体，避免过大引起不必要的渲染开销。具体视锥体大小可以自适应于场景，也可以自适应于视角。

- 2、截取 Shadow Depth Map。将 Shadow Depth Map 保存为适当尺寸的纹理。尺寸越大，包含的信息越多，结果越精确，消耗的内存越多，相反尺寸越小，精度越低，会引发锯齿问题。

- 3、确定转换矩阵。因为截取的 Shadow Depth Map 是在光源空间下的，对于场景中任意一点，需要知道其在光源空间中的位置。与视图空间的矩阵转换类似，分别需要模型矩阵、视图矩阵和投影矩阵。首先通过模型矩阵将模型所在的局部坐标系转换为世界坐标系。然后通过视图矩阵将世界坐标系转化为相机坐标系。最后通过投影矩阵将相机坐标系转换为正交投影坐标系。

- 4、比较深度。通过在着色器中计算矩阵转换后，比较物体某一点的深度与 Shadow Depth Map 中该点的值来确定是否处于阴影中。

基础的 Shadow Map 其结果还有待改进，此时通过 Shadow Map 渲染阴影会出现条纹装的阴影，发生如图 5-3 所示的 Shadow Acne 现象。

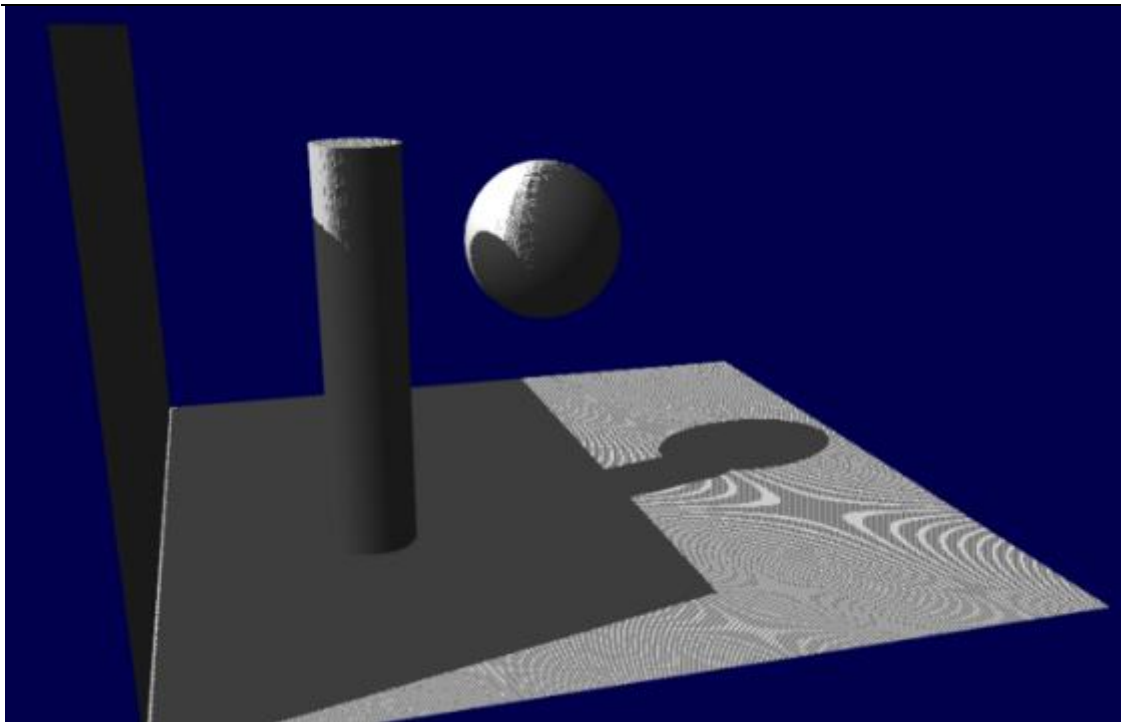


图 5-3 Shadow Acne

（图片来自互联网 OpenGL_3_3_Tutorial_Translation-Github）

因为 Shadow Depth Map 的分辨率不够，因此多个像素会对应 Map 上的同一个点。由于 Map 是对场景的离散取样，所以在实际场景中，深度会产生阶梯状变化，导致有一部分比实际场景中的深度要大，这一部分不会产生阴影，而另一部分比实际场景中的深度要小，这部分会产生阴影，因此会出现条纹状的阴影。这是因为物体的实际深度与自己的采样深度不同导致的，即实际深度大于采样深度。这种自己采样的副本遮挡住了自己实际的物体的现象，称为自阴影（Self Shadowing）。为了解决自阴影，我们需要在计算实际深度的时候，将其往灯光方向拉近一点，让其减小一个值即可，也就是添加 Shadow Bias。但是 Shadow Bias 的值要尽量小，如果 Shadow Bias 过大，会导致物体计算影子时的深度与他的实际深度差别过大，导致影子和物体产生分离。

通常 Shadow Bias 的计算方法基于物体坡度，其中坡度由顶点的法向量得到：

$$Bias = factorSlope \times slope + constantBias \quad (5-10)$$

需要注意的是法向量需要转换到光源空间坐标系下。

因为我们的判定条件是 0-1 的，其边缘没有过度，这种无渐变的影子，被称为硬阴影（Hard Shadow），与之相对的，有边缘渐变效应的则称为软阴影^[46]（Soft Shadow）。

这里我们使用 Percentage Closer Filtering（简称 PCF）算法来生成软阴影。PCF 是常用于柔化 Shadow Map 边缘，产生软阴影的一种技术，最早来源于 1987 年 Computer Graphics 上的论文^[47]，因为算法原理简单，实现快速而且并不会占用太多运算，所以我们选择使用 PCF 算法来生成软阴影。PCF 的核心思想是通过周围的点判断该点的边缘程度。如果是影子内部，则它周围的点也一定在阴影之中，如果是影子边缘，则它周围的点会有一些不在阴影中，且越是边缘的位置，不在阴影中的邻居点越多。对每个点通过泊松分部来进行 4x4 的采样，最后插值算出影子强度。

最终基于多边形模型对云的体积场生成密度估计函数，结合由 Value 噪声对体积云的体积场进行建模，使用改进的基于模型表面的光线步进算法进行渲染，以及使用 Shadow Map 加速阴影渲染，实现体积云在移动端的加速。

5.6 实验结果

本章在通过基于模型的体积云建模以及基于模型表面的光线步进算法进行体积云渲染，大幅提高体积云在移动端的渲染效率，实现了体积云在移动端流畅的实时渲染。因为性能的限制，目前还没有一个成熟的方法能够在移动端实现体积云的实时渲染，主流方法仍然在使用平面云或模型云，这些方法有着易穿帮、无法穿过云层等缺点。

本文方法在移动端部署，使用系统为 Android 系统，处理器为 8 核高通骁龙 845，内存大小为 8G，使用 Unreal 平台版本为 4.22.3，使用 HLSL 和 C++ 进行编

写。

在渲染效率方面,目前还没有一个成熟的方法能够在移动端实现体积云的渲染,大多数方法仍然在使用平面云或者模型云。通过 Schneider^[16]的方法在移动端渲染一帧的时间在 200ms 以上,完全无法实现实时渲染。而通过本文的方法,使用基于多边形模型的方法进行体积云的模拟,通过密度估计函数确定体积云的密度,并使用改进的基于模型表面的光线步进算法,大幅减少采样步数,通过使用 Shadow Map 简化阴影渲染等方法,在牺牲一定视觉效果的情况下,大幅提高渲染效率,最终渲染一帧的时间在 30ms 左右,能够达到稳定 30 帧每秒的效率渲染,在移动端实现流畅的体积云实时渲染。

在视觉效果方面,我们将本文提出的移动端体积云实时渲染方法与《Sky 光·遇》^[48]相比较,该游戏在 2017Apple 秋季新品发布会上首次展示,凭借其云中翱翔的震撼视觉效果赢得满堂喝彩,在移动端云的模拟方面有着优秀的视觉效果,经过研究其使用的是模型云方法,利用 3D 噪点贴图,将模型沿着法线方向挤压进行多层渲染,通过后处理使光线更加鲜艳,对云的视觉效果进行优化,如图 5-4~5-9 所示。为了方便比较,模型部分仿照《Sky 光·遇》场景生成相似的粗糙模型。这里同样我们更关注体积云渲染方法本身,我们主要关注云的边缘的通透效果、自阴影效果以及太阳光穿过云时的表现。可以看到在这些方面,我们的移动端体积云渲染方法表现良好,最终实现了在移动端能够流畅运行的体积云模拟方法。



图 5-4 移动端加速的体积云云海的视觉效果



图 5-5 《Sky 光·遇》中云海的视觉效果



图 5-6 移动端加速的体积云自阴影的视觉效果



图 5-7 《Sky 光·遇》中云的自阴影视觉效果



图 5-8 移动端加速的体积云翻涌的视觉效果



图 5-9 《Sky 光·遇》中云翻涌的视觉效果

5.7 本章小结

本章主要介绍体积云在移动端加速。首先介绍超纹理，然后详细探讨基于超纹理概念使用模型对体积云体积场生成密度估计函数的方法，并与基于 Value 噪声生成的湍流噪声相结合完成对体积云体积场的建模，接下来在此基础上介绍基于模型表面的光线步进加速算法，介绍使用 Shadow Map 对阴影渲染的加速方法，最后对结果进行展示。

第6章 总结和展望

6.1 本文工作总结

云是一种常见的自然景观，而云的渲染一直是计算机图形学领域中一个重要的组成部分。本文主要研究了体积云的建模和渲染以及移动端的加速方法，具体工作主要有以下几个方面：

1、提出了一种基于噪声生成的体积云建模的方法，本文在实现体积云建模的基础上，对体积云建模进行优化。该方法基于 Perlin 噪声和 Worley 噪声生成的分型噪声，同时结合密度-高度函数和时间函数生成体积场，对云进行建模。该方法建模出的云具有更多细节，更接近于现实中自然的云，同时可以对多种不同的云进行建模并且可以实现动态变化。

2、提出了一种基于光线步进的体积云渲染的方法，在实现体积云渲染的基础上，对体积云渲染进行优化。该方法结合比尔-朗伯定律、暗边效应、相位函数等对体积云光照进行近似计算，使用改进的光线步进技术对云进行实时渲染。该方法渲染出的云效果逼真，光照更加真实，可以模拟云的运动，也可以穿过云层产生交互。

3、提出了一种体积云在移动端加速的方法，实现体积云在移动端的实时渲染。该方法基于模型对云的体积场生成密度估计函数和基于 Value 噪声生成的湍流噪声对体积云的体积场进行建模，使用改进的基于模型表面的光线步进算法进行渲染，使用 Shadow Map 加速阴影渲染。该方法可以在移动端实现较为逼真的动态体积云的实时渲染，可以模拟云的运动，可以穿过浅层云层产生交互。

6.2 未来研究工作

本文主要研究了动态体积云的建模和实时渲染及移动端加速方法，取得了良

好的效果。但是仍然存在一些可以改进和优化的方面：

1、本文中对于体积云的建模主要类型针对于低云族的层积云、层云和积云，后续可以考虑把方法推广到高云族的卷云、卷积云等更多类型的云，对其进行建模。

2、本文中对于体积云的渲染主要针对于光照模型的计算和算法本身的优化，后续可以考虑采用分帧渲染等一些硬件上的方法来对渲染进行加速^{[49][50]}，比如将显示器分为多个区域，如 4×4 ，在每一帧中渲染其中一个区域，其他区域对上一帧进行重用。

3、本文对于体积云在移动端加速方法所使用的密度估计函数在特定情况下会与实际情况有所偏差，还有着很大的进一步优化空间。

参考文献

- [1] Scholl I, Suder S, Schiffer S. Direct Volume Rendering in Virtual Reality[J]. 2018.
- [2] Harris M J, Lastra A. Real-time cloud rendering for games[C]//Proceedings of Game Developers Conference. 2002: 21-29.
- [3] Harris M J, Lastra A. Real-time cloud rendering[C]//Computer Graphics Forum. Oxford, UK and Boston, USA: Blackwell Publishers Ltd, 2001, 20(3): 76-85.
- [4] Harris M J. Real-time cloud simulation and rendering[D]. University of North Carolina at Chapel Hill, 2003.
- [5] Wang N. Realistic and fast cloud rendering[J]. Journal of graphics tools, 2004, 9(3): 21-40.
- [6] Blinn J F. Light reflection functions for simulation of clouds and dusty surfaces[C]//Acm Siggraph Computer Graphics. ACM, 1982, 16(3): 21-29.
- [7] Blinn J, Gomez J, Max N, et al. The simulation of natural phenomena (panel session)[C]//ACM SIGGRAPH Computer Graphics. ACM, 1983, 17(3): 137-139.
- [8] Perlin K. An image synthesizer[J]. ACM Siggraph Computer Graphics, 1985, 19(3): 287-296.
- [9] Gardner G Y. Visual simulation of clouds[C]//ACM Siggraph Computer Graphics. ACM, 1985, 19(3): 297-304.
- [10] Nishita T, Dobashi Y, Nakamae E. Display of clouds taking into account multiple anisotropic scattering and sky light[C]//Proceedings of the 23rd annual conference on Computer graphics and interactive techniques. ACM, 1996: 379-386.
- [11] Dobashi Y, Kaneda K, Yamashita H, et al. A simple, efficient method for realistic animation of clouds[C]//Proceedings of the 27th annual conference on Computer graphics and interactive techniques. ACM Press/Addison-Wesley Publishing Co., 2000: 19-28.
- [12] Miyazaki R, Yoshida S, Dobashi Y, et al. A method for modeling clouds based on atmospheric fluid dynamics[C]//Proceedings Ninth Pacific Conference on Computer Graphics and Applications. Pacific Graphics 2001. IEEE, 2001: 363-372.
- [13] Bouthors A, Neyret F, Lefebvre S, et al. Real-time realistic illumination and shading

- of stratiform clouds[C]. Eurographics Workshop on Natural Phenomena, 2006: 41-50.
- [14]Bouthors A, Neyret F, Max N, et al. Interactive multiple anisotropic scattering in clouds[C]//Proceedings of the 2008 symposium on Interactive 3D graphics and games. ACM, 2008: 173-182.
- [15]Yusov E. High-Performance Rendering of Realistic Cumulus Clouds Using Pre-computed Lighting[C]//High Performance Graphics. 2014: 127-136.
- [16]Schneider A, Vos N. The real-time volumetric cloudscapes of horizon: Zero dawn[J]. Advances in Real-Time Rendering in Games, ACM SIGGRAPH, 2015.
- [17]Schneider A. Real-time volumetric cloudscapes[J]. GPU Pro 7: Advanced Rendering Techniques, 2016, 97.
- [18]Elhaddad A, Elhaddad F, Sheng B, et al. Real-time cloud simulation using lennard-jones approximation[C]//Proceedings of the 29th International Conference on Computer Animation and Social Agents. ACM, 2016: 131-137.
- [19]Nowak Ł, Bąk A, Czajkowski T, et al. Modeling and Rendering of Volumetric Clouds in Real-Time with Unreal Engine 4[C]//International Conference on Computer Vision and Graphics. Springer, Cham, 2018: 68-78.
- [20]Hufnagel R, Held M. A survey of cloud lighting and rendering techniques[C]. International Conference in Central Europe on Computer Graphics and Visualization, 2012: 205-216.
- [21]Dobashi Y, Iwasaki K, Yue Y, et al. Visual simulation of clouds[J]. Visual Informatics, 2017, 1(1): 1-8.
- [22]任威, 梁晓辉, 马上, 沈旭昆. 大规模三维云实时模拟方法[J]. 计算机辅助设计与图形学学报, 2010, 22(04): 662-669.
- [23]陈小龙, 袁赞. 基于体绘制的动态云场景模拟[J]. 计算机技术与发展, 2015, 25(02): 29-32.
- [24]刘芳, 刘贤梅, 吴琼. 三维场景中体积云的研究与实现[J]. 计算机技术与发展, 2015, 25(05): 135-137+142.
- [25]柏靖云. 动态积云绘制算法的实现与分析[J]. 现代计算机(专业版), 2018(08): 85-88+93.

- [26]Schpok J, Simons J, Ebert D S, et al. A real-time cloud modeling, rendering, and animation system[C]//Proceedings of the 2003 ACM SIGGRAPH/Eurographics symposium on Computer animation. Eurographics Association, 2003: 160-166.
- [27]Max N. Optical models for direct volume rendering[J]. IEEE Transactions on Visualization and Computer Graphics, 1995, 1(2): 99-108.
- [28]Elinas P, Stürzlinger W. Real-time rendering of 3d clouds[J]. Journal of Graphics Tools, 2000, 5(4): 33-45.
- [29]毛星云. 基于物理的渲染 (PBR) 白皮书 [OL]. <https://zhuanlan.zhihu.com/p/53086060/>
- [30]Burley B, Studios W D A. Physically-based shading at disney[C]//ACM SIGGRAPH. 2012, 2012: 1-7.
- [31]Burley B. Extending the Disney BRDF to a BSDF with integrated subsurface scattering[J]. the Physically Based Shading in Theory and Practice SIGGRAPH Course, 2015.
- [32]Montes R, Ureña C. An overview of BRDF models[J]. University of Grenada, Technical Report LSI-2012, 2012, 1.
- [33]Cornette W M, Shanks J G. Physically reasonable analytic expression for the single-scattering phase function[J]. Applied optics, 1992, 31(16): 3152-3160.
- [34]Henyey L G, Greenstein J L. Diffuse radiation in the galaxy[J]. The Astrophysical Journal, 1941, 93: 70-83.
- [35]Unreal engine 官网[OL]. <https://www.unrealengine.com/>
- [36]Perlin K. Making noise[J]. GDC Talk, 1999, 1999.
- [37]Perlin K. Improving noise[C]//ACM transactions on graphics (TOG). ACM, 2002, 21(3): 681-682.
- [38]Worley S. A cellular texture basis function[C]//Proceedings of the 23rd annual conference on Computer graphics and interactive techniques. ACM, 1996: 291-294.
- [39]Zhou K, Ren Z, Lin S, et al. Real-time smoke rendering using compensated ray marching[C]//ACM Transactions on Graphics (TOG). ACM, 2008, 27(3): 36.
- [40]Upton C, Keeler M. V-buffer: visible volume rendering[J]. ACM SIGGRAPH Computer Graphics, 1988, 22(4): 59-64.

-
- [41] Drebin R A, Carpenter L, Hanrahan P. Volume rendering[C]//ACM Siggraph Computer Graphics. ACM, 1988, 22(4): 65-74.
- [42] Perlin K, Hoffert E M. Hypertexture[C]//ACM Siggraph Computer Graphics. ACM, 1989, 23(3): 253-262.
- [43] Lokovic T, Veach E. Deep shadow maps[C]//Proceedings of the 27th annual conference on Computer graphics and interactive techniques. ACM Press/Addison-Wesley Publishing Co., 2000: 385-392.
- [44] Stamminger M, Drettakis G. Perspective shadow maps[C]//ACM transactions on graphics (TOG). ACM, 2002, 21(3): 557-562.
- [45] Kolivand H, Sunar M S. An overview on base real-time shadow techniques in virtual environments[J]. Telkomnika, 2012, 10(1): 171.
- [46] Hasenfratz J M, Lapierre M, Holzschuch N, et al. A survey of real-time soft shadows algorithms[C]//Computer Graphics Forum. Oxford, UK and Boston, USA: Blackwell Publishing, Inc, 2003, 22(4): 753-774.
- [47] Reeves W T, Salesin D H, Cook R L. Rendering antialiased shadows with depth maps[C]//ACM Siggraph Computer Graphics. ACM, 1987, 21(4): 283-291.
- [48] 《Sky 光·遇》官网[OL]. <https://sky.163.com/>
- [49] Harris M J, Baxter W V, Scheuermann T, et al. Simulation of cloud dynamics on graphics hardware[C]//Proceedings of the ACM SIGGRAPH/EUROGRAPHICS conference on Graphics hardware. Eurographics Association, 2003: 92-101.
- [50] Raphael L S, Jeferson V R, el al. Multiple-Volume Volumetric Rendering Using Graphic Accelerator[C]//CLEI, 2018, 612-621

攻读硕士学位期间主要的研究成果

[1] Jiang M, Lan W, Chang J, et al. A game prototype for understanding the safety issues of a lifeboat launch[J]. Virtual Reality, 2018, 22(2): 137-148.

致谢

首先要感谢我的导师童若锋老师，在老师的悉心指导和督促下才使得论文得以顺利完成。同时也感谢实验室和寝室的同学对我的帮助，两年半的相处时光，期间给予我许多帮助，也对我的论文提了许多建议，给了我很多启发。

在研究生两年半中还得到众多老师的关心支持和帮助。在此，谨向老师们致以衷心的感谢和崇高的敬意。最后，我要向百忙之中对本文进行审阅，评议和参与本人答辩的各位老师表示感谢。